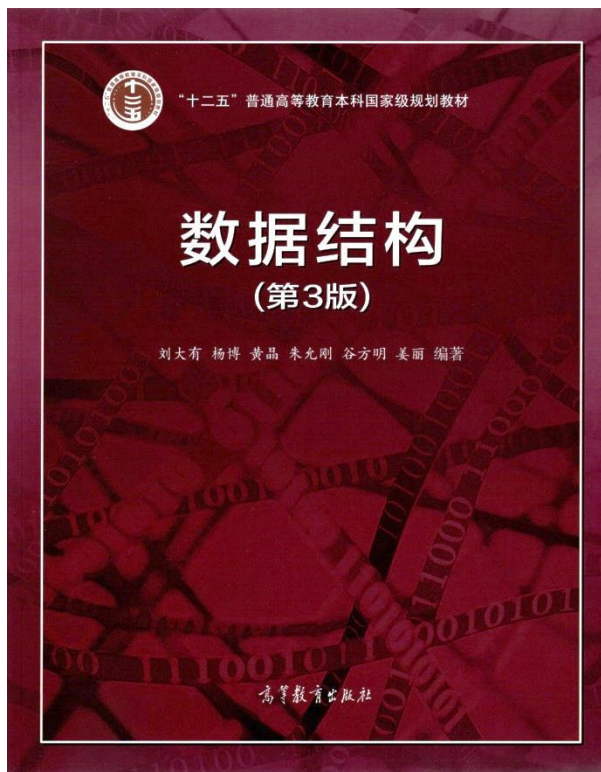




数组与矩阵

- 数组存储与寻址
- 特殊矩阵的压缩存储
- 稀疏矩阵的压缩存储
- 动态规划初探
- 子集生成

数据之法
结构之美
算法之道

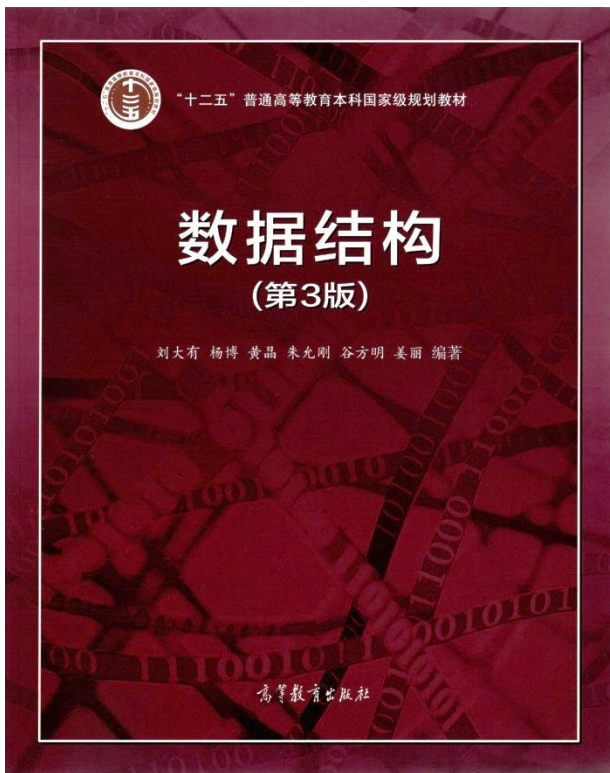
zhuyungang@jlu.edu.cn

我觉得就跟龟兔赛跑一样
如果兔子没睡着
乌龟无论怎么努力
都不可能赢得了兔子
但是乌龟后来明白
它一直往前跑
它不是为了赢过兔子
是为了想去他要去的的地方



慕课自学内容（必看，计入期末成绩）

自学内容	视频时长	主讲人
数组的存储和寻址	1分40秒	杨博教授
一维数组类	20分58秒	杨博教授
矩阵类	22分35秒	杨博教授



数组与矩阵

- 数组存储与寻址回顾
- 特殊矩阵的压缩存储
- 稀疏矩阵的压缩存储
- 动态规划初探
- 子集生成

数据之法
结构之美
算法之道

zhuyungang@jlu.edu.cn

n 维数组（按行优先存储）

- 各维元素个数 $m_1, m_2, m_3, \dots, m_n$, 每个元素占 C 个存储单元
- 数组元素 $a[i_1][i_2]\dots[i_n]$ 的存储地址:

$$LOC(i_1, i_2, \dots, i_n) = LOC(0, \dots, 0) + (i_1 \times m_2 \times m_3 \times \dots \times m_n + i_2 \times m_3 \times m_4 \times \dots \times m_n + i_3 \times m_4 \times \dots \times m_n + \dots + i_{n-1} \times m_n + i_n) \times C$$

$$LOC(0, \dots, 0) + \sum_{k=1}^n \left(i_k \times \prod_{p=k+1}^n m_p \right) \times C$$

例子

➤ 已知数组A[3][5][11][3]

➤ 给出按行优先存储下的A[i][j][k][l]地址计算公式

$$\text{Loc}(A) + (i * 5 * 11 * 3 + j * 11 * 3 + k * 3 + l) * C$$

$$= \text{Loc}(A) + (165i + 33j + 3k + l) * C$$

课下练习

已知三维数组 $A[10][20][15]$ 以按行优先方式存储，每个元素占2个存储单元。若元素 $A[0][0][0]$ 的存储地址是1000，则 $A[8][5][10]$ 的存储地址是5970。【吉林大学计软两院22级期末考试题】

n 维数组（按列优先存储）

- 各维元素个数 $m_1, m_2, m_3, \dots, m_n$, 每个元素占 C 个存储单元
- 数组元素 $a[i_1][i_2]\dots[i_n]$ 的存储地址:

$$LOC(i_1, i_2, \dots, i_n) = LOC(0, \dots, 0) + (i_1 + i_2 \times m_1 + i_3 \times m_1 \times m_2 + \dots + i_{n-1} \times m_1 \times \dots \times m_{n-2} + i_n \times m_1 \times \dots \times m_{n-1}) \times C$$

$$LOC(0, \dots, 0) + \sum_{k=2}^n \left(i_1 + i_k \times \prod_{p=1}^{k-1} m_p \right) \times C$$

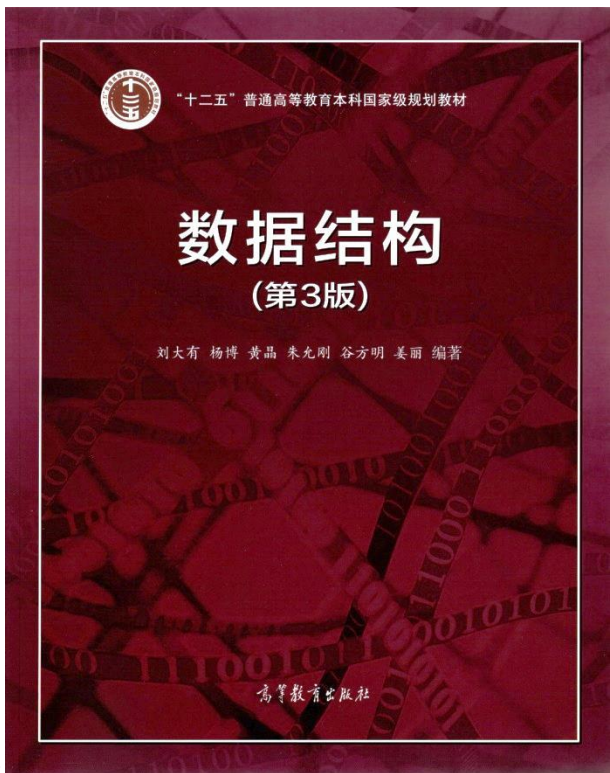
例子

➤ 已知数组A[3][5][11][3]

➤ 给出按列优先存储下的A[i][j][k][l]地址计算公式

$$\text{Loc}(A) + (i + j*3 + k*3*5 + l*3*5*11)*C$$

$$= \text{Loc}(A) + (i + 3j + 15k + 165l)*C$$



数组与矩阵

- 数组存储与寻址回顾
- **特殊矩阵的压缩存储**
- 稀疏矩阵的压缩存储
- 动态规划初探
- 子集生成

数据之法
结构之美
算法之道

zhuyungang@jlu.edu.cn

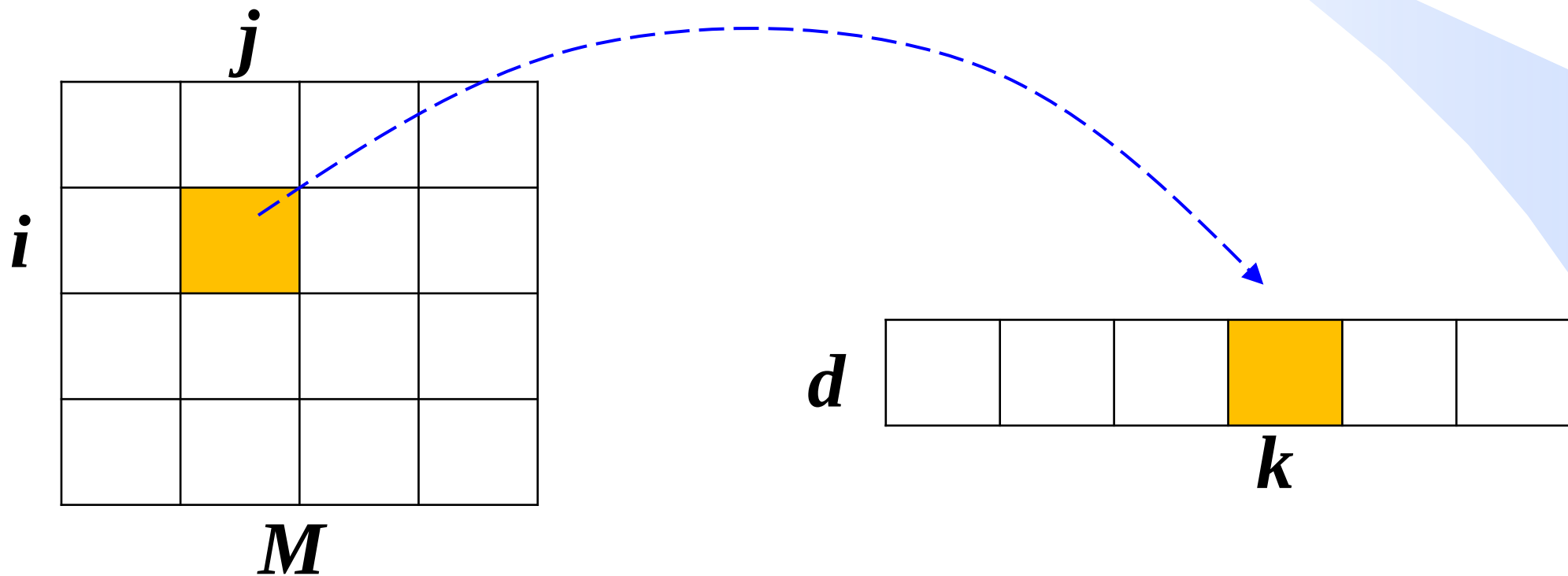
对角矩阵的压缩存储

- 若 $n \times n$ 的方阵 M 是对角矩阵，则对所有的 $i \neq j$ ($1 \leq i, j \leq n$) 都有 $M(i, j) = 0$ ，即非对角线上的元素均为0，非0元素只在对角线上。
- 对于一个 $n \times n$ 的对角矩阵，至多只有 n 个非0元素，因此只需存储 n 个对角元素。
- 可采用一维数组 $d[]$ 来压缩存储对角矩阵。

$$\begin{pmatrix} A_1 & & & O \\ & A_2 & & \\ & & \ddots & \\ O & & & A_l \end{pmatrix}$$

特殊矩阵的压缩存储需考虑2个问题

- 需要多大存储空间：数组 $d[]$ 需要多少元素
- 地址映射：矩阵的任意元素 $M(i, j)$ 在 $d[]$ 中的位置（下标），即把矩阵元素的下标映射到数组 d 的下标



对角矩阵的压缩存储

➤ 用一维数组 $d[n]$ 存储对角矩阵，其中 $d[i-1]$ 存储 $M(i, i)$ 的值。

$$M = \begin{bmatrix} a_{11} & & & \\ & a_{22} & & \\ & & a_{33} & \\ & & & a_{44} \end{bmatrix}$$

$$d = [d_0 \quad d_1 \quad d_2 \quad d_3]$$

$$M(i, j) = \begin{cases} d[i-1], & i = j \\ 0, & i \neq j \end{cases}$$

三角矩阵的压缩存储

- 三角矩阵分为上三角矩阵和下三角矩阵。
- 方阵M是上三角矩阵，当且仅当*i>j*时有 $M(i, j)=0$ 。
- 方阵M是下三角矩阵，当且仅当*i<j*时有 $M(i, j)=0$ 。

$$U = \begin{bmatrix} u_{1,1} & u_{1,2} & u_{1,3} & \dots & u_{1,n} \\ & u_{2,2} & u_{2,3} & \dots & u_{2,n} \\ \vdots & & \ddots & \ddots & \vdots \\ & (0) & & \ddots & u_{n-1,n} \\ 0 & & \dots & & u_{n,n} \end{bmatrix}$$

$$L = \begin{bmatrix} l_{1,1} & & \dots & & 0 \\ l_{2,1} & l_{2,2} & & (0) & \\ l_{3,1} & l_{3,2} & \ddots & & \vdots \\ \vdots & \vdots & \ddots & \ddots & \\ l_{n,1} & l_{n,2} & \dots & l_{n,n-1} & l_{n,n} \end{bmatrix}$$

三角矩阵的压缩存储

以下三角矩阵**M**为例，讨论其压缩存储方法：

➤将下三角矩阵以按行优先方式压缩存放在一维数组**d**

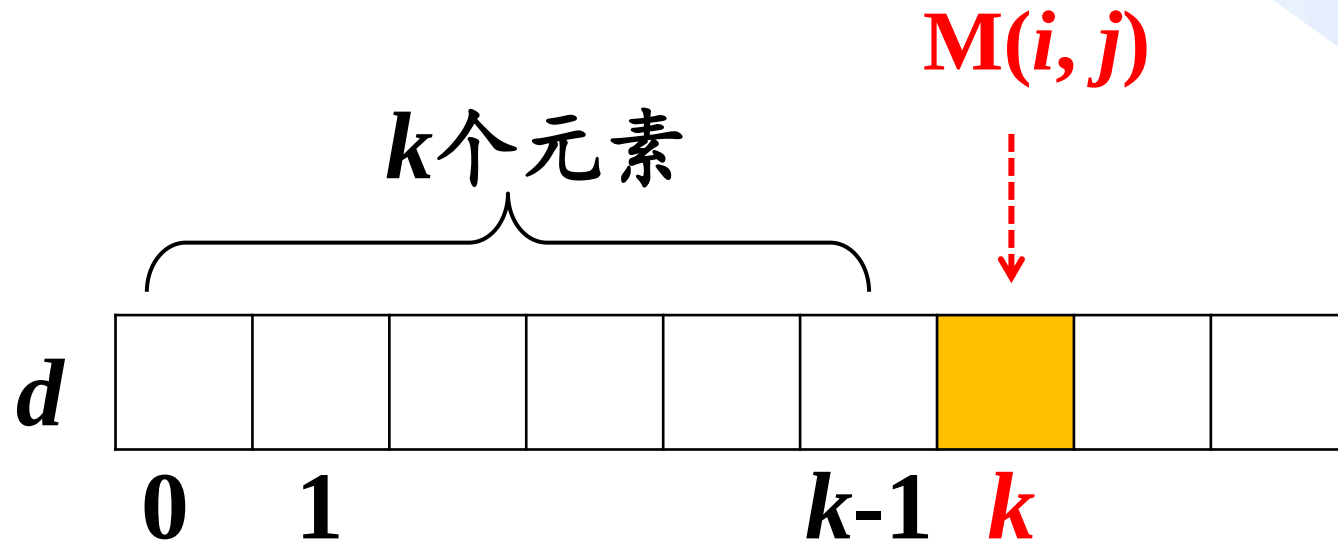
✓**d**需要多少个元素？ $n(n+1)/2$

✓**M(i, j)**在数组**d**的什么位置？

$$\mathbf{L} = \begin{bmatrix} l_{1,1} & & \cdots & & 0 \\ l_{2,1} & l_{2,2} & & (0) & \\ l_{3,1} & l_{3,2} & \ddots & & \vdots \\ \vdots & \vdots & \ddots & \ddots & \\ l_{n,1} & l_{n,2} & \cdots & l_{n,n-1} & l_{n,n} \end{bmatrix}$$

$M(i, j)$ 在数组d的什么位置

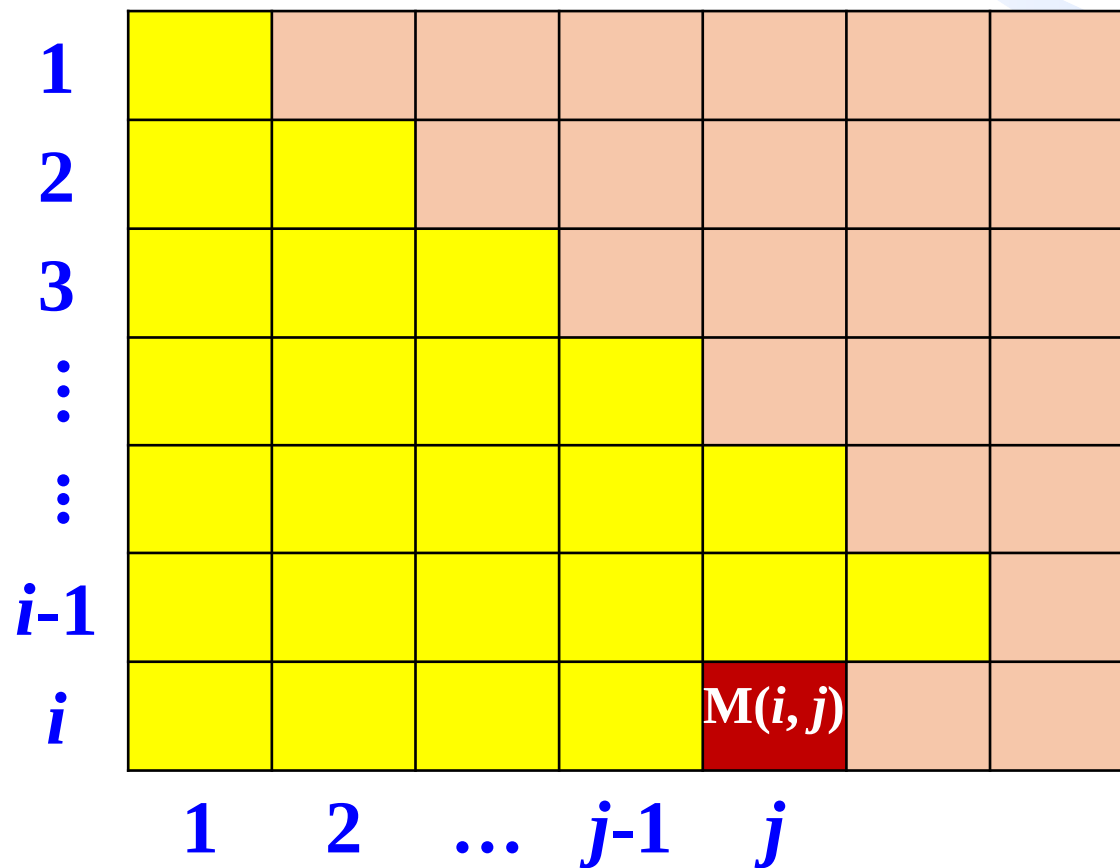
➤ 若元素 $M(i, j)$ 前面有 k 个元素，则 $M(i, j)$ 存储在 $d[k]$ 位置



下三角矩阵的压缩存储

➤ 设元素 $M(i, j)$ 前面有 k 个元素，可以计算出

➤ $k = 1 + 2 + \dots + (i - 1) + (j - 1) = i(i - 1)/2 + (j - 1)$



1						
2						
3						
⋮						
⋮						
$i-1$						
i				$M(i, j)$		
	1	2	...	$j-1$	j	

下三角矩阵的压缩存储

➤ 设元素 $M(i, j)$ 前面有 k 个元素，可以计算出

➤ $k = 1 + 2 + \dots + (i - 1) + (j - 1) = i(i - 1)/2 + (j - 1)$

➤ $M(i, j) = d[k] = d[i(i - 1)/2 + (j - 1)]$

$$M(i, j) = \begin{cases} d[i(i - 1)/2 + (j - 1)], & i \geq j \\ 0, & i < j \end{cases}$$

对称矩阵M的压缩存储

- 方阵 $M_{n \times n}$ 是对称矩阵，当且仅当对于任何 $1 \leq i, j \leq n$ ，均有 $M(i, j) = M(j, i)$ 。
- 因为对称矩阵中 $M(i, j)$ 与 $M(j, i)$ 的信息相同，所以只需存储M的下三角部分的元素信息。
- 将对称矩阵存储到一维数组d
- d需要多少个元素？ $n(n+1)/2$
- $M(i, j)$ 的寻址方式是什么？

➤ $i \geq j$, $M(i, j) = d[k]$, $k = i(i-1)/2 + (j-1)$

对于对称矩阵中的下三角元素 $M(i, j)$ ($i \geq j$)，和下三角矩阵压缩存储的映射公式一样

对于上三角元素 $M(i, j)$ ($i < j$)，元素值与下三角矩阵中的元素 $M(j, i)$ 相同

➤ $i < j$, $M(i, j) = M(j, i) = d[q]$, $q = j(j-1)/2 + (i-1)$

$$M(i, j) = \begin{cases} d[i(i-1)/2 + (j-1)], & i \geq j \\ d[j(j-1)/2 + (i-1)], & i < j \end{cases}$$

课下思考

设矩阵 A 是一个对称矩阵，为了节省存储空间，将其下三角部分按照行优先存放在一维数组 $B[0, \dots, n(n+1)/2-1]$ 中，对于下三角部分中的任一元素 $a_{i,j}$ ($i \geq j$, i 和 j 从1开始取值)，在一维数组 B 中的下标 K 的值是_____。【百度校园招聘笔试题】

A. $i(i-1)/2+j-1$

B. $i(i+1)/2+j$

C. $i(i+1)/2+j-1$

D. $i(i-1)/2+j$

课下思考

设有一个 12×12 的对称矩阵 M ，将其上三角元素 $M(i, j)$ ($1 \leq i, j \leq 12$)按行优先存入C语言的一维数组 N 中，则元素 $M(6, 6)$ 在 N 中的下标是_____。【2018年考研题全国卷】

M 第一行12个元素，

第二行11个元素，

第三行10个元素，

第四行9个元素，

第五行8个元素，

$M(6, 6)$ 是第6行第1个元素，

即前面有50个元素，故 $M(6, 6) = N[50]$

$$U = \begin{bmatrix} u_{1,1} & u_{1,2} & u_{1,3} & \dots & u_{1,n} \\ & u_{2,2} & u_{2,3} & \dots & u_{2,n} \\ \vdots & & \ddots & \ddots & \vdots \\ & & & \ddots & u_{n-1,n} \\ & & \dots & & u_{n,n} \end{bmatrix}$$

课下思考

设有一个 10×10 的对称矩阵 M ，将其上三角元素 $M(i, j)$ ($1 \leq i, j \leq 10$)按列优先存入C语言的一维数组 N 中，则元素 $M_{7,2}$ 在 N 中的下标是_____。【2020年考研题全国卷】

$$M(7, 2) = N[22]$$

三对角矩阵M的压缩存储

方阵 $M_{n \times n}$ 中任意元素 $M(i, j)$, 当 $|i - j| > 1$ 时, 有 $M(i, j) = 0$, 则 M 称为三对角矩阵。

$$M = \begin{bmatrix} a_{11} & a_{12} & & \\ a_{21} & a_{22} & a_{23} & \\ & a_{32} & a_{33} & a_{34} \\ & & a_{43} & a_{44} \end{bmatrix}$$

三对角矩阵M的压缩存储

$a_{1,1}$	$a_{1,2}$						
$a_{2,1}$	$a_{2,2}$	$a_{2,3}$					
	$a_{3,2}$	$a_{3,3}$	$a_{3,4}$				
		$a_{4,3}$	$a_{4,4}$	$a_{4,5}$			
			...	...			
				$a_{i,i-1}$	$a_{i,i}$	$a_{i,i+1}$	
					...	...	
					$a_{n-1,n-2}$	$a_{n-1,n-1}$	$a_{n-1,n}$
						$a_{n,n-1}$	$a_{n,n}$

$$M(i,j) = \begin{cases} d[2i+j-3], & |i-j| \leq 1 \\ 0, & |i-j| > 1 \end{cases}$$

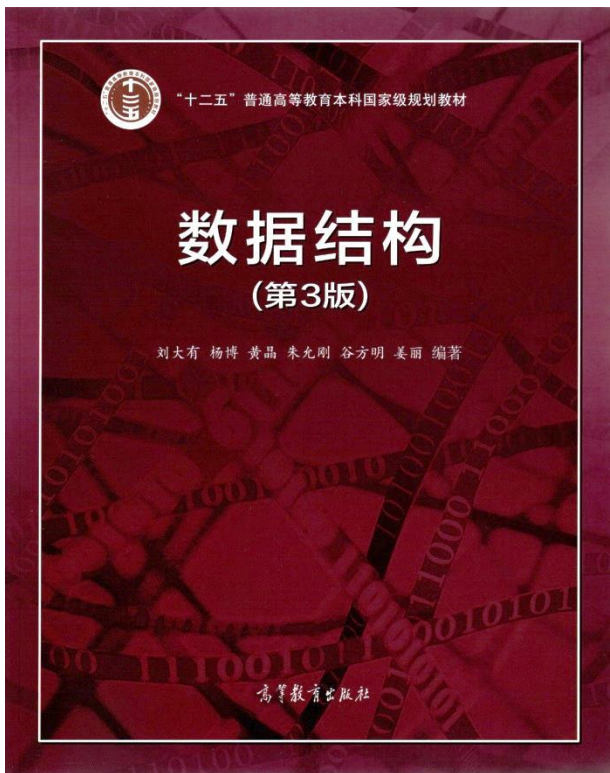
M(i,j)前面有k个元素
 $k = 2+(i-2)*3+(j-i)+1$
 $=2i+j-3$

$a_{1,1}$	$a_{1,2}$	$a_{2,1}$	$a_{2,2}$	$a_{2,3}$	...	$a_{n-1,n}$	$a_{n,n-1}$	$a_{n,n}$
-----------	-----------	-----------	-----------	-----------	-----	-------------	-------------	-----------

课下思考

有一个100阶的三对角矩阵M，其元素 $M(i, j)$ ($1 \leq i, j \leq 100$)按行优先依次压缩存入下标从0开始的一维数组N中，则元素 $M(30, 30)$ 在N中的下标是_____。【2016年考研题全国卷】

$$2i + j - 3 = 2 * 30 + 30 - 3 = 87$$



数组与矩阵

- 数组存储与寻址
- 特殊矩阵的压缩存储
- **稀疏矩阵的压缩存储**
- 动态规划初探
- 子集生成

数据之法
结构之美
算法之道

zhuyungang@jlu.edu.cn

稀疏矩阵的压缩存储

- 若矩阵A非零元素的个数远小于零元素的个数，则称A为稀疏矩阵。
 - ✓ 压缩存储：仅存储非零元素，节省空间。
 - ✓ 特点：零元素分布一般没有规律，难以通过公式实现压缩存储的地址映射。
- 用三元组结点来存储矩阵的每个非零元素 a_{ij} ，其中 i 和 j 为元素的行号和列号：

i	j	a_{ij}
-----	-----	----------
- 如何在三元组结点的基础上实现对整个稀疏矩阵的存储？
 - ✓ 顺序存储方式实现：三元组表
 - ✓ 链接存储方式实现：十字链表

三元组表

将表示稀疏矩阵的**非零元素**的三元组结点以**按行优先**顺序存储于数组，称之为三元组表。

稀疏矩阵

$$A = \begin{pmatrix} 50 & 0 & 0 & 0 \\ 10 & 0 & 20 & 0 \\ 0 & 5 & 0 & 0 \\ -30 & 0 & -60 & 0 \end{pmatrix}$$

三元组表

B[0]	1	1	50
B[1]	2	1	10
B[2]	2	3	20
B[3]	3	2	5
B[4]	4	1	-30
B[5]	4	3	-60

```
struct Triple{
    int row;
    int col;
    int value;
};
Triple B[100];
```

三元组表

A

若采用三元组表存储稀疏矩阵 M ，除三元组及 M 包含的非零元素个数外，下列数据中还需要保存的是_____。【2023年考研题全国卷】

- I. M 的行数 II. M 中包含非零元素的行数
III. M 的列数 IV. M 中包含非零元素的列数

- A. 仅 I、III B. 仅 I、IV C. 仅 II、IV D. I、II、III、IV

稀疏矩阵的转置操作

转置前

$$A = \begin{pmatrix} 50 & 0 & 0 & 0 \\ 10 & 0 & 20 & 0 \\ 0 & 0 & 0 & 0 \\ -30 & 0 & -60 & 5 \end{pmatrix}$$

$a[0]$	1	1	50
$a[1]$	2	1	10
$a[2]$	2	3	20
$a[3]$	4	1	-30
$a[4]$	4	3	-60
$a[5]$	4	4	5

转置后

$$B = \begin{pmatrix} 50 & 10 & 0 & -30 \\ 0 & 0 & 0 & 0 \\ 0 & 20 & 0 & -60 \\ 0 & 0 & 0 & 5 \end{pmatrix}$$

$b[0]$	1	1	50
$b[1]$	1	2	10
$b[2]$	1	4	-30
$b[3]$	3	2	20
$b[4]$	3	4	-60
$b[5]$	4	4	5

稀疏矩阵的转置操作

a

转置前

1	1	50
2	1	10
2	3	20
4	1	-30
4	3	-60
4	4	5

b

转置后

1	1	50
1	2	10
1	4	-30
3	2	20
3	4	-60
4	4	5

稀疏矩阵的转置算法

```

void Transpose(Triple a[], int m, int n, int t, Triple b[]){
//将三元组表a表示的m行n列矩阵转置,存于三元组表b中,a中非0元素个数为t
    int j = 0; //j标识当前填三元组b的第几位
    if (t == 0) return; //a空
    for (int k = 1; k <= n; k++) //填转置后的矩阵的第k行的元素
        for (int i = 0; i < t; i++) //扫描矩阵a,看哪些元素列号是k
            if (a[i].col == k) { //看a的哪些元素列号是k
                b[j].row = k; //转置后行号应为k
                b[j].col = a[i].row; //列号应为其在a中的行号
                b[j].value = a[i].value;
                j++; //赋值三元组表b中的下一个结点
            }
    }
}

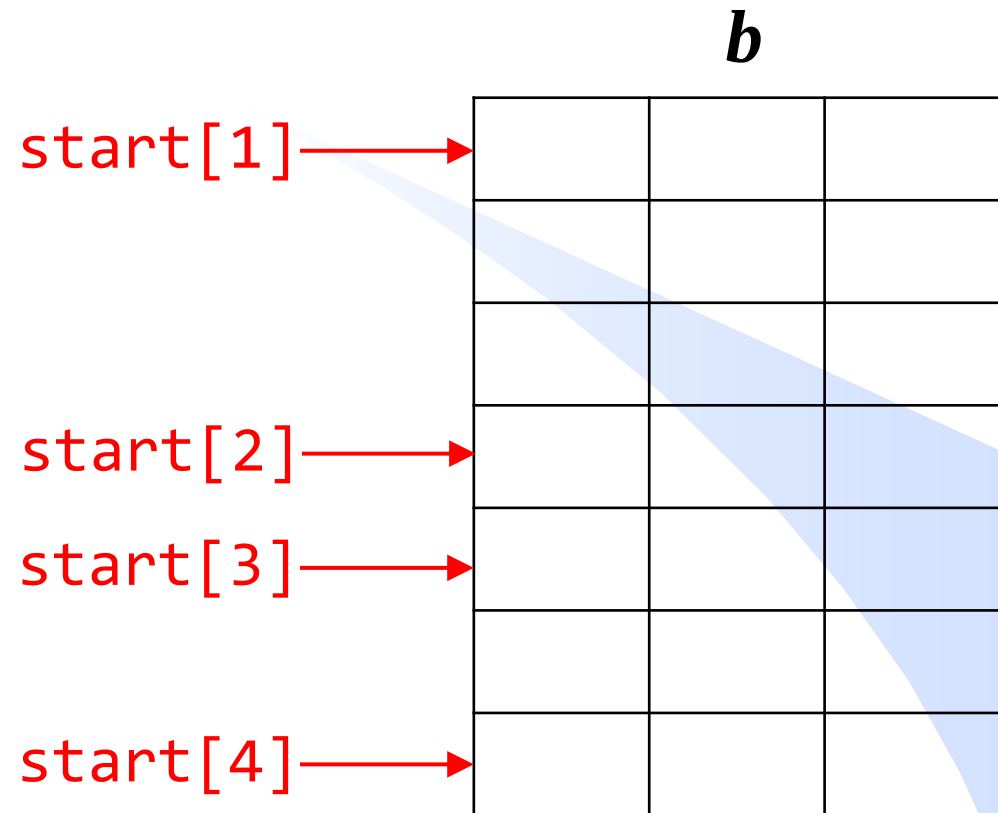
```

时间复杂度
 $O(nt)$

稀疏矩阵的快速转置算法

a

1	1	50
2	1	10
2	3	20
3	2	5
4	1	-30
4	3	6
4	4	5



start[1]=0
start[2]=start[1]+转置后第1行的元素个数
start[i]=start[i-1]+转置后第i-1行的元素个数

rowsize[]:长度为 n , 存放转置后各行非0元素的个数,
即转置前各列非0元素的个数。

```
for(int i = 1; i <= n; i++)
    rowsize[i] = 0;
```

```
for(int i = 0; i < t; i++) {
    k = a[i].col;
    rowsize[k]++;
}
```

1	2	3	4

1	1	50
2	1	10
2	3	20
3	2	5
4	1	-30
4	3	6
4	4	5

$start[]$: 长度为 n , 存放转置后的矩阵每行在三元组表 b 中的起始位置。

B

```
start[1] = 0;
for(int i = 2; i <= n; i++)
    start[i] = start[i-1] + rowsize[i-1];
for(int i = 0; i < t; i++) {
    k = a[i].col;
    j = start[k];
    b[j].row = a[i].col;
    b[j].col = a[i].row;
    b[j].value = a[i].value;
    start[k]++;
}
```


b

1	1	50
2	1	10
2	3	20
3	2	5
4	1	-30
4	3	6
4	4	5

a

时间复杂度 $O(n+t)$

稀疏矩阵的三元组表存储方式分析

- 节省空间，但对于非零元的**位置或个数**经常发生变化的矩阵运算就显得不太适合。
- 如：矩阵某些位置频繁的加上或减去一个数，使有的元素由0变成非0，由非0变成0。导致三元组表频繁进行插入删除操作，需要频繁元素移动。

十字链表

	1	2	3	4
1	0	0	6	0
2	4	0	0	0
3	0	9	0	7
4	0	0	0	8

<i>left</i>		<i>up</i>
<i>row</i>	<i>col</i>	<i>value</i>

```

struct ListNode{
    int row;           //数据
    int col;           //该结点所在行
    int value;         //该结点所在列
    ListNode *left;    //指向左侧相邻非零元素的指针
    ListNode *up;      //指向上方相邻非零元素的指针
};
    
```

十字链表

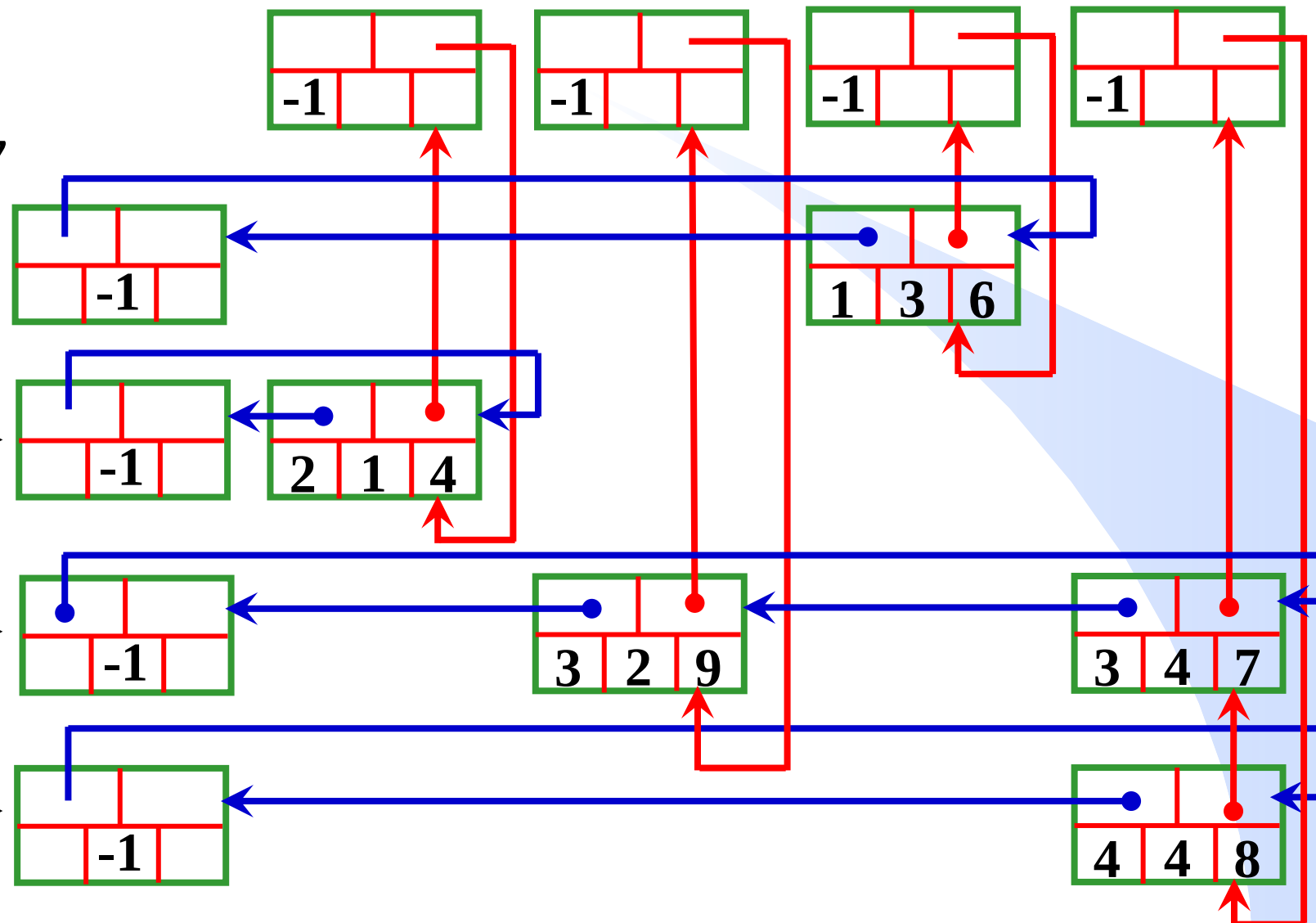
B

left		up	
row	col	value	

	1	2	3	4
1	0	0	6	0
2	4	0	0	0
3	0	9	0	7
4	0	0	0	8

headCol

headRow



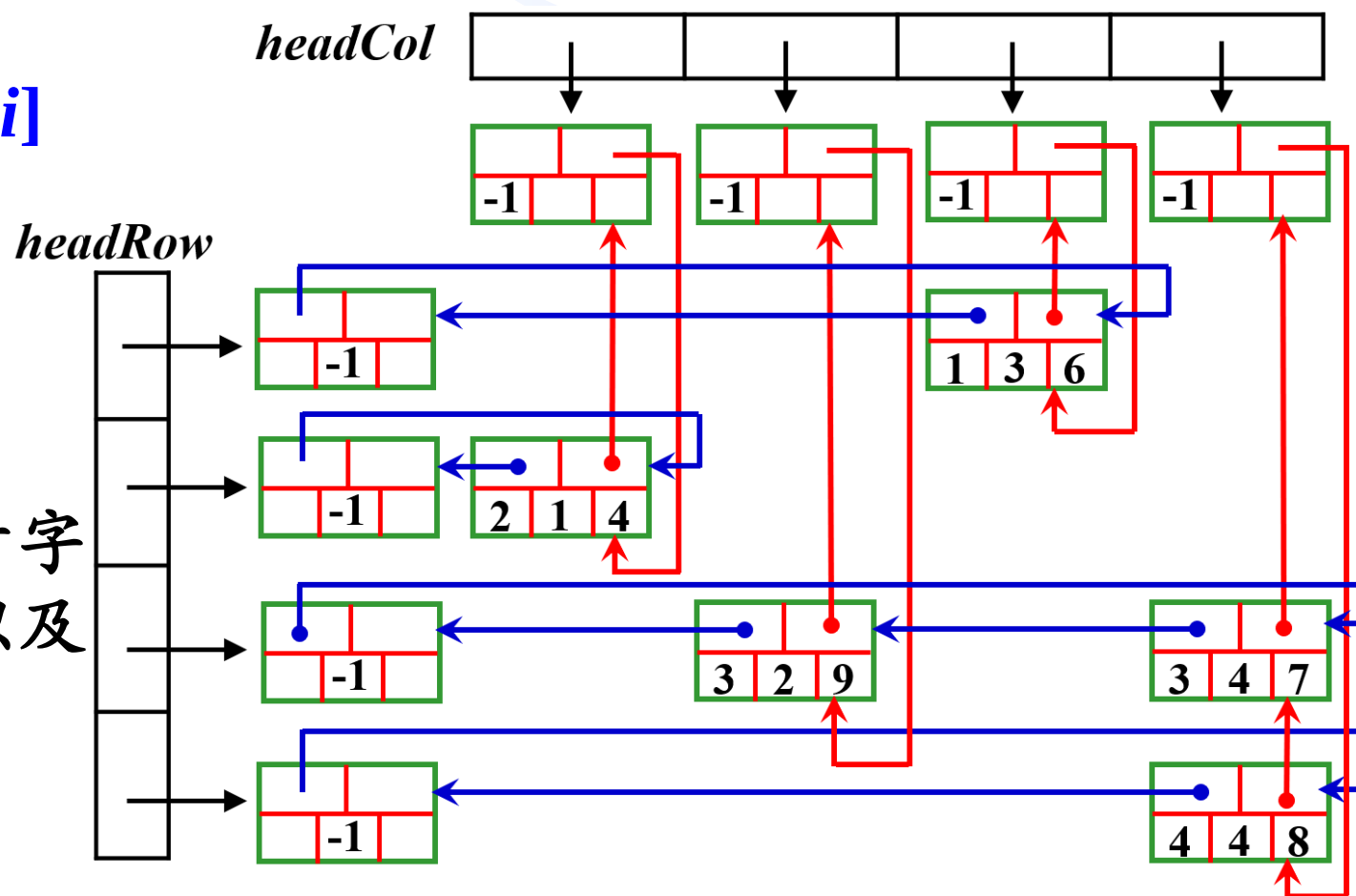
十字链表

矩阵的每一行、列都设置为由一个哨位结点引导的循环链表

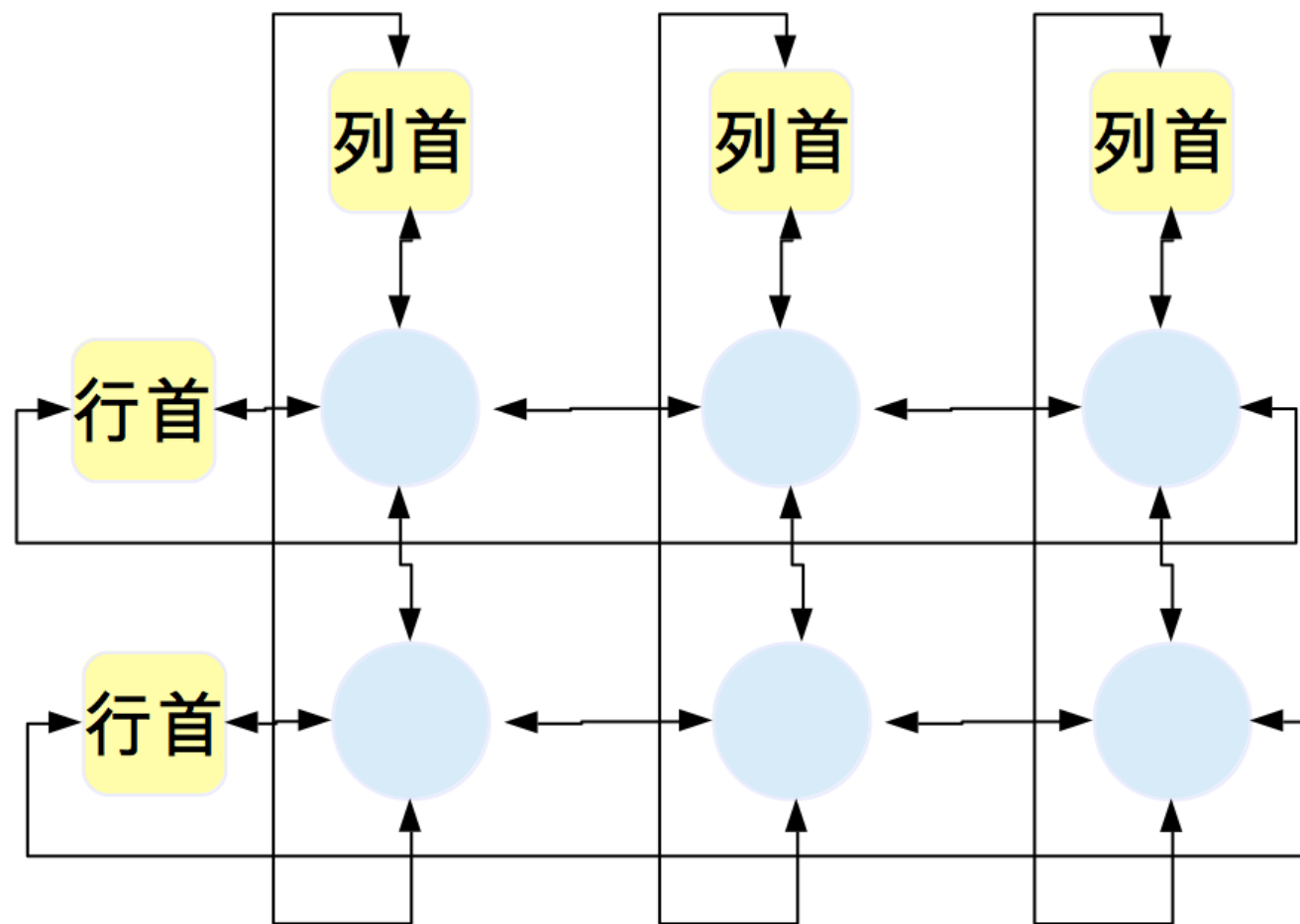
- $headRow[i]$ 是第 i 行链表的头指针，指向第 i 行链表的哨位结点
- $headCol[j]$ 是第 j 列链表的头指针，指向第 j 列链表的哨位结点
- 若第 i 行无非零元素，则
 $headRow[i] \rightarrow left == headRow[i]$

- 若第 j 列无非零元素，则
 $headCol[j] \rightarrow up == headCol[j]$

- 对矩阵的运算实质上就是在十字链表中插入结点、删除结点以及改变某个结点的数据域的值。



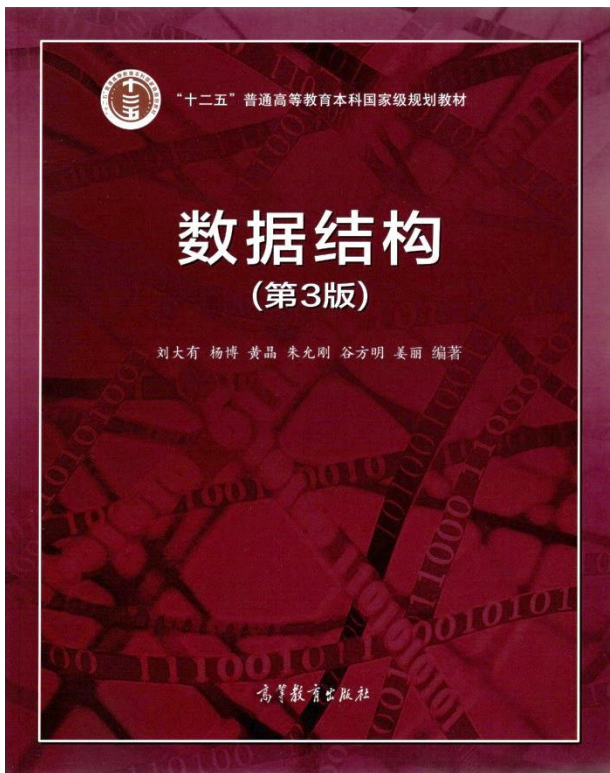
双向十字链表（舞蹈链Dancing Links）



Donald Knuth
 斯坦福大学教授
 图灵奖获得者
 美国科学院院士
 美国工程院院士

移除一个元素

```
p->left->right = p->right;
P->right->left = p->left;
P->up->down = p->down;
P->down->up = p->up;
```



数组与矩阵

- 数组存储与寻址
- 特殊矩阵的压缩存储
- 稀疏矩阵的压缩存储
- **动态规划初探**
- 子集生成

数据之法
结构之美
算法之道

zhuyungang@jlu.edu.cn

递归算法求解斐波那契数列

$$F(n) = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ F(n-1) + F(n-2) & n \geq 2 \end{cases}$$

```
int F(int n){  
    if(n<=1) return n;  
    return F(n-1)+F(n-2);  
}
```



Fibonacci
(1170年—1250年)
意大利数学家

时间复杂度

```
int F(int n){
    if(n<=1) return n;
    return F(n-1)+F(n-2);
}
```

$$T(n) = \begin{cases} 0 & n \leq 1 \\ 1 & n = 2 \\ T(n-1) + T(n-2) + 1 & n \geq 2 \end{cases}$$

$$T(n-1) = T(n-2) + T(n-3) + 1$$

$$\begin{aligned} T(n) &\leq 2T(n-1) \\ &\leq 2^2T(n-2) \\ &\leq 2^3T(n-3) \\ &\leq 2^4T(n-4) \\ &\dots \\ &\leq 2^{n-2}T(2) \\ &= 2^{n-2} \end{aligned}$$

$$T(n) = O(2^n)$$

时间复杂度

```
int F(int n){
    if(n<=1) return n;
    return F(n-1)+F(n-2);
}
```

$$T(n) = \begin{cases} 0 & n \leq 1 \\ 1 & n = 2 \\ T(n-1) + T(n-2) + 1 & n \geq 2 \end{cases}$$

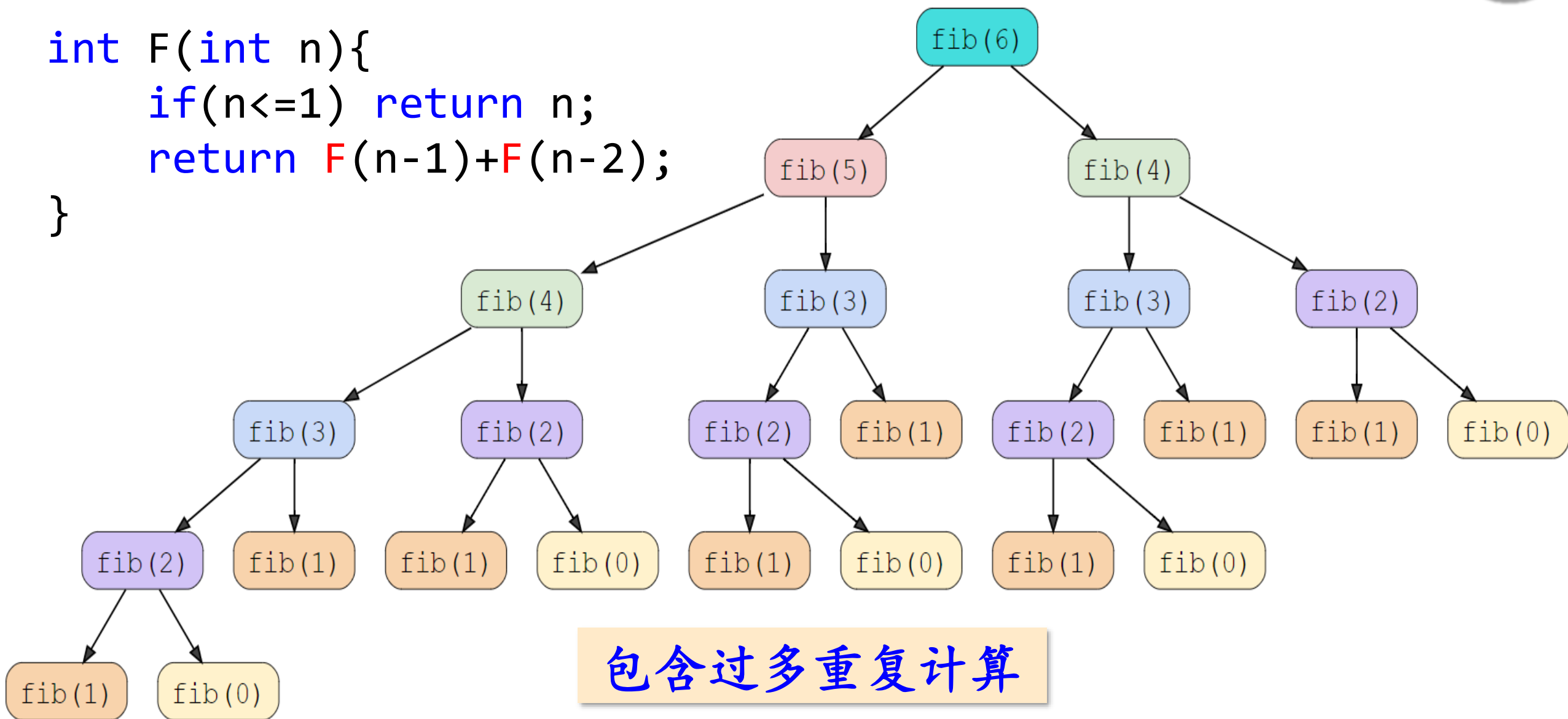
$$T(n-1) = T(n-2) + T(n-3) + 1$$

$$\begin{aligned} T(n) &\geq 2T(n-2) \\ &\geq 2^2T(n-4) \\ &\geq 2^3T(n-6) \\ &\geq 2^4T(n-8) \\ &\dots \\ &\geq 2^{(n-2)/2}T(2) \\ &= 2^{(n-2)/2} \end{aligned}$$

$$T(n) = \Omega(2^{n/2})$$

计算效率低的原因

```
int F(int n){  
    if(n<=1) return n;  
    return F(n-1)+F(n-2);  
}
```



动态规划 (*Dynamic Programming, DP*) 的提出者



Richard Bellman
(1920-1984)

南加州大学教授
美国科学院院士
美国工程院院士

递推

```
int F[N];
int Fib(int n){
    F[0]=0; F[1]=1;
    for(int i=2; i<=n; i++)
        F[i]=F[i-1]+F[i-2];
    return F[n];
}
```

时间复杂度 $O(n)$

空间复杂度 $O(n)$

$$F(n) = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ F(n-1) + F(n-2) & n \geq 2 \end{cases}$$

从前往后
当算到 $F[i]$ 时,
 $F[i-1]$ 和 $F[i-2]$
已经算完了

递归vs递推

F	0	1	1	2	3	5	8	13	21	34	55
	0	1	2	3	4	5	6	7	8	9	10

空间优化

```
int Fib(int n){  
    int cur,prev0=0,prev1=1;  
    for(int i=2; i<=n; i++){  
        cur = prev0+prev1;  
        prev0 = prev1;  
        prev1 = cur;  
    }  
    return cur;  
}
```

空间复杂度
 $O(1)$

	prev0	prev1	cur		
--	-------	-------	-----	--	--

动态规划 (Dynamic Programming, DP)

- 将一个复杂的问题分解成若干个子问题，通过综合子问题的解来得到原问题的解。
- 自底向上先求解最小的子问题，并把结果存储在表格中，在求解大的子问题时直接从表格中查询小的子问题的解，以避免重复计算，从而提高效率。
- 往往可以通过“递推”来实现。

跳台阶问题

一个台阶总共有 n 层。如果一只青蛙可以一次跳1层，也可以跳2层。编写算法对于给定的 n ，计算出青蛙跳到最顶层总共有多少种跳法。【大厂面试题[LeetCode70](#)】

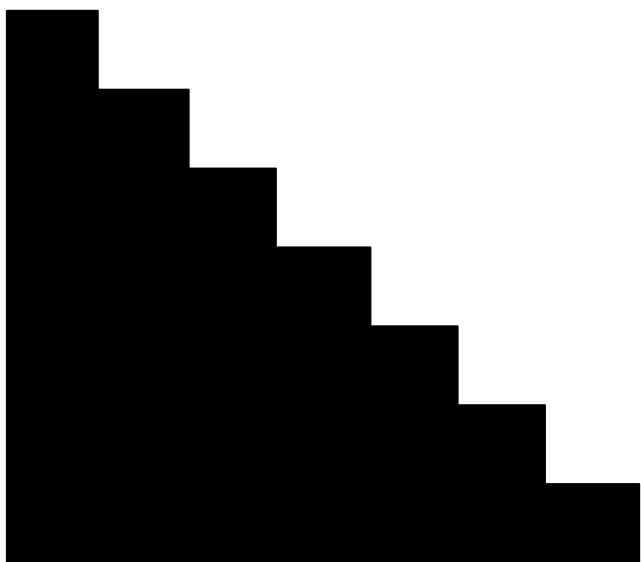


$$F(n) = \begin{cases} 1 & n = 1 \\ 2 & n = 2 \\ F(n-1) + F(n-2) & n > 2 \end{cases}$$

课下思考

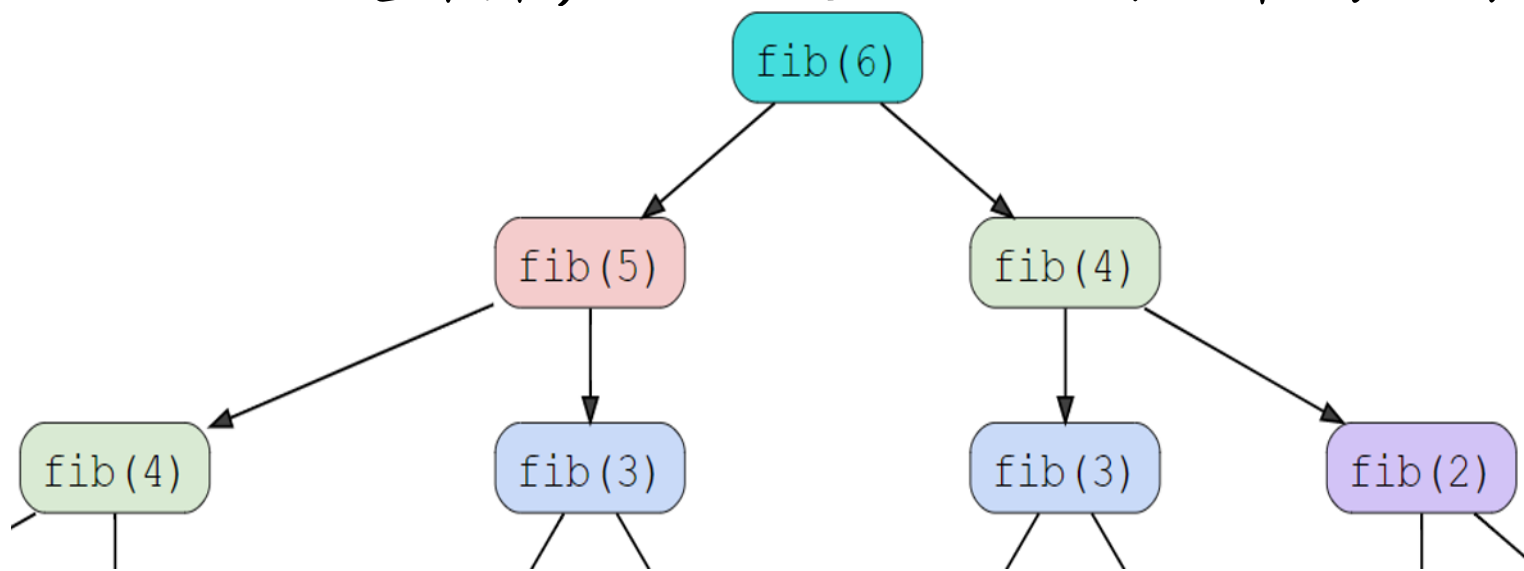
一个台阶总共有 n 层。如果一只青蛙可以一次跳1层，也可以跳2层，也可以跳3层。编写算法对于给定的 n ，计算出青蛙跳到最顶层总共有多少种跳法。

$$F(n) = \begin{cases} 1 & n = 1 \\ 2 & n = 2 \\ 4 & n = 3 \\ F(n-1) + F(n-2) + F(n-3) & n > 3 \end{cases}$$



什么问题可以用动态规划方法高效解决

- **最优子结构**：问题的最优解所包含的子问题的解也是最优的。大问题的解包含子问题的解，可以通过子问题的解推导出大问题的解。
- **无后效性**：将每个子问题的求解过程作为一个阶段，当前阶段的求解只与之前阶段有关，而与之后的阶段无关。
- **重叠子问题**：求解过程中每次产生的子问题并不总是新问题，有大量子问题重复。动态规划在处理重叠子问题时，每个子问题只计算一次，从而避免重复计算，这也是动态规划效率高的原因。



使用动态规划方法求解问题的一般过程

- ① **寻找子问题**：把原问题分解成若干子问题，子问题和原问题形式相似，只不过规模小一些，例如从原来的 n 变成 $n-1$ 。
- ② **定义状态**：和子问题相关的各个变量的的一组取值称为一个状态。
- ③ **导出递推公式**：从一个或多个已知状态的值，推导出未知状态的值，递推公式也称“状态转移方程”。
- ④ **确定边界条件**：确定递推的终止条件和边界条件。

如何找子问题、如何定义状态、如何推导出递推公式，没有固定的规则，需要具体问题具体分析。多做题，熟能生巧。

二维情况

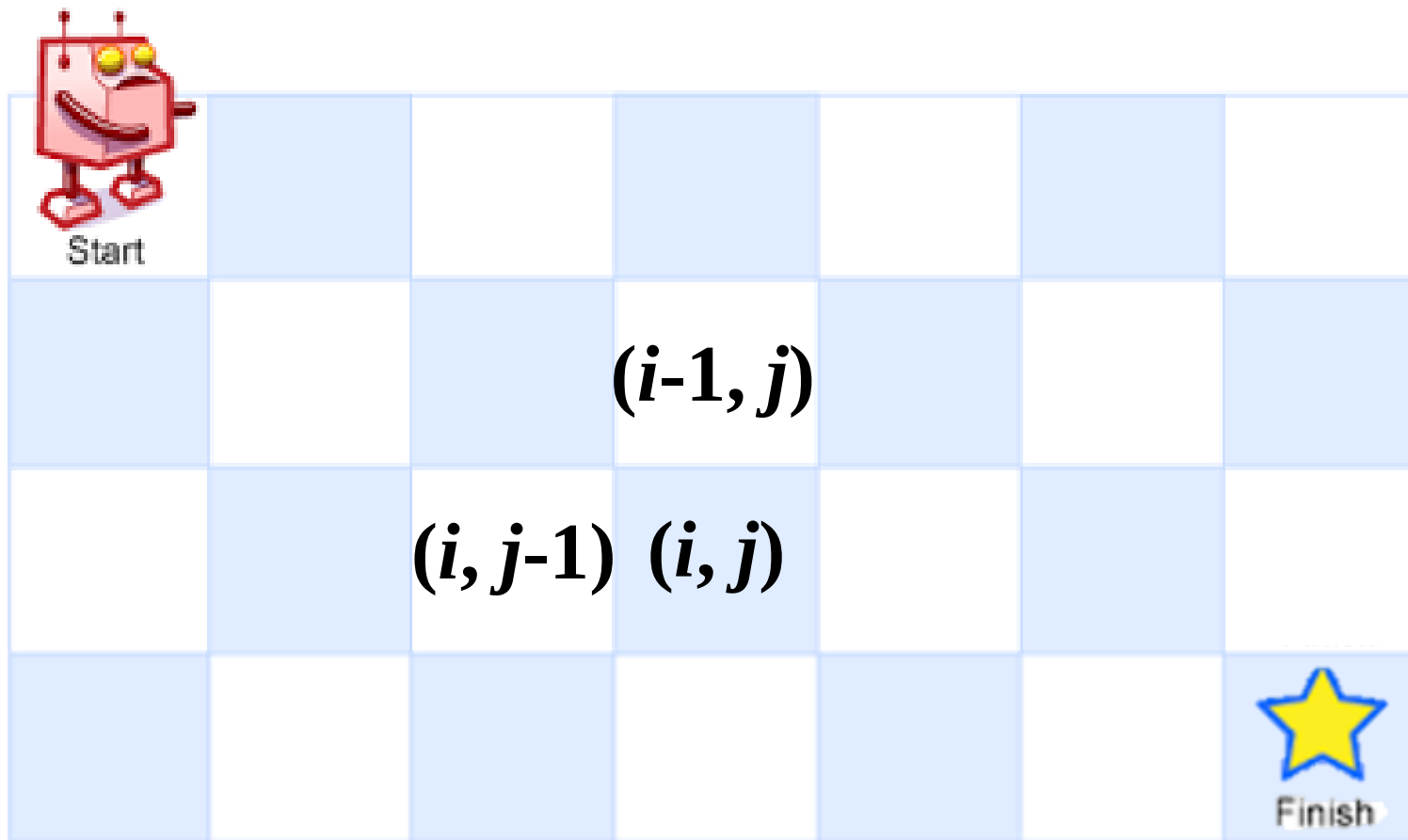
一个机器人位于一个 $m \times n$ 网格的左上角，每次只能**向下**或者**向右**移动一步。机器人试图达到网格的右下角，编写程序计算总共有多少条不同的路径。【大厂面试题[LeetCode62](#)】



令 $F(i, j)$ 表示起点到点 (i, j) 的路径条数

从起点到点 (i, j) 只有两种途径：

- ①先到点 $(i-1, j)$, 再向下走一步
- ②先到点 $(i, j-1)$, 再向右走一步



$$F(i, j) = \begin{cases} 1 & i = 0 \text{ 或 } j = 0 \\ F(i, j - 1) + F(i - 1, j) & \text{其他} \end{cases}$$

递归算法

$$F(i, j) = \begin{cases} 1 & i = 0 \text{ 或 } j = 0 \\ F(i, j-1) + F(i-1, j) & \text{其他} \end{cases}$$

初始调用 $F(m-1, n-1)$

```
int F(int i, int j){  
    if(i==0 || j==0) return 1;  
    return F(i, j-1)+F(i-1, j);  
}
```

指数级复杂度

动态规划算法

$$F(i, j) = \begin{cases} 1 & i = 0 \text{ 或 } j = 0 \\ F(i, j - 1) + F(i - 1, j) & \text{其他} \end{cases}$$

		F[i-1][j]	
<i>i</i>	F[i][j-1]	F[i][j]	

动态规划算法

B

$$F(i, j) = \begin{cases} 1 & i = 0 \text{ 或 } j = 0 \\ F(i, j - 1) + F(i - 1, j) & \text{其他} \end{cases}$$

$n-1$

$m-1$

			$F[m-2][n-1]$
		$F[m-1][n-2]$	$F[m-1][n-1]$

动态规划算法

$$F(i, j) = \begin{cases} 1 & i = 0 \text{ 或 } j = 0 \\ F(i, j - 1) + F(i - 1, j) & \text{其他} \end{cases}$$

1	1	1	1
1	2	3	4
1			
1			
1			

动态规划算法

B

```
#define N 110
int uniquePaths(int m, int n)
{
    int F[N][N]={0};
    for(int j=0; j<n; j++)
        F[0][j] = 1; //填第0行
    for(int i=0; i<m; i++)
        F[i][0] = 1; //填第0列
    for(int i=1; i<m; i++)
        for(int j=1; j<n; j++)
            F[i][j] = F[i-1][j] + F[i][j-1];
    return F[m-1][n-1];
}
```

1	1	1	1
1			
1		$F[i-1][j]$	
1	$F[i][j-1]$	$F[i][j]$	
1			

时间复杂度 $O(mn)$
空间复杂度 $O(mn)$

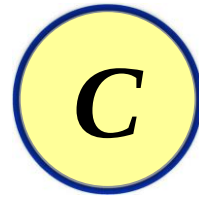
空间优化

		$F[i-1][j]$	
	$F[i][j-1]$	$F[i][j]$	

空间优化

		$F[i-1][j]$	$F[i-1][j+1]$
	$F[i][j-1]$	$F[i][j]$	$F[i][j+1]$

空间优化



	<i>b</i>	<i>d</i>	<i>f</i>
<i>a</i>	<i>c</i>	<i>e</i>	<i>g</i>

$$c = a + b$$

$$e = c + d$$

$$g = e + f$$

空间优化——滚动数组

$i = 1$

1	1	1	1
---	---	---	---

空间优化——滚动数组

$i = 1$

1	2	1	1
---	---	---	---

空间优化——滚动数组

$i = 1$

1	2	3	1
---	---	---	---

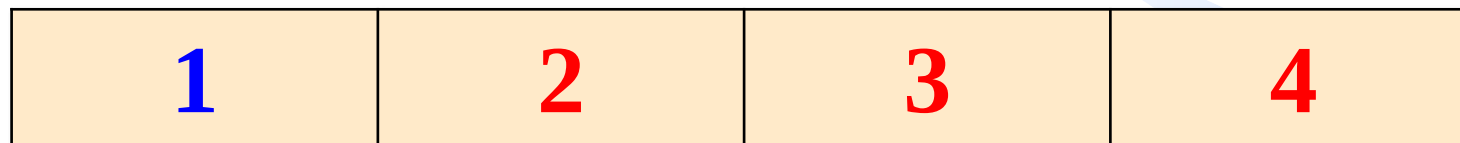
空间优化——滚动数组

$i = 1$

1	2	3	4
---	---	---	---

空间优化——滚动数组

$i = 2$



空间优化——滚动数组

$i = 2$

1	3	3	4
---	---	---	---

空间优化——滚动数组

$i = 2$

1	3	6	4
---	---	---	---

空间优化——滚动数组

$i = 2$

1	3	6	10
---	---	---	----

空间优化——滚动数组

$i = 2$

1	3	6	10
---	---	---	----

```
#define N 110
int uniquePaths(int m, int n){
    int F[N]={0};
    for(int i=0; i<n; i++)
        F[i] = 1;
    for(int i=1; i<m; i++)
        for (int j=1; j<n; j++)
            F[j] = F[j-1] + F[j];
    return F[n-1];
}
```

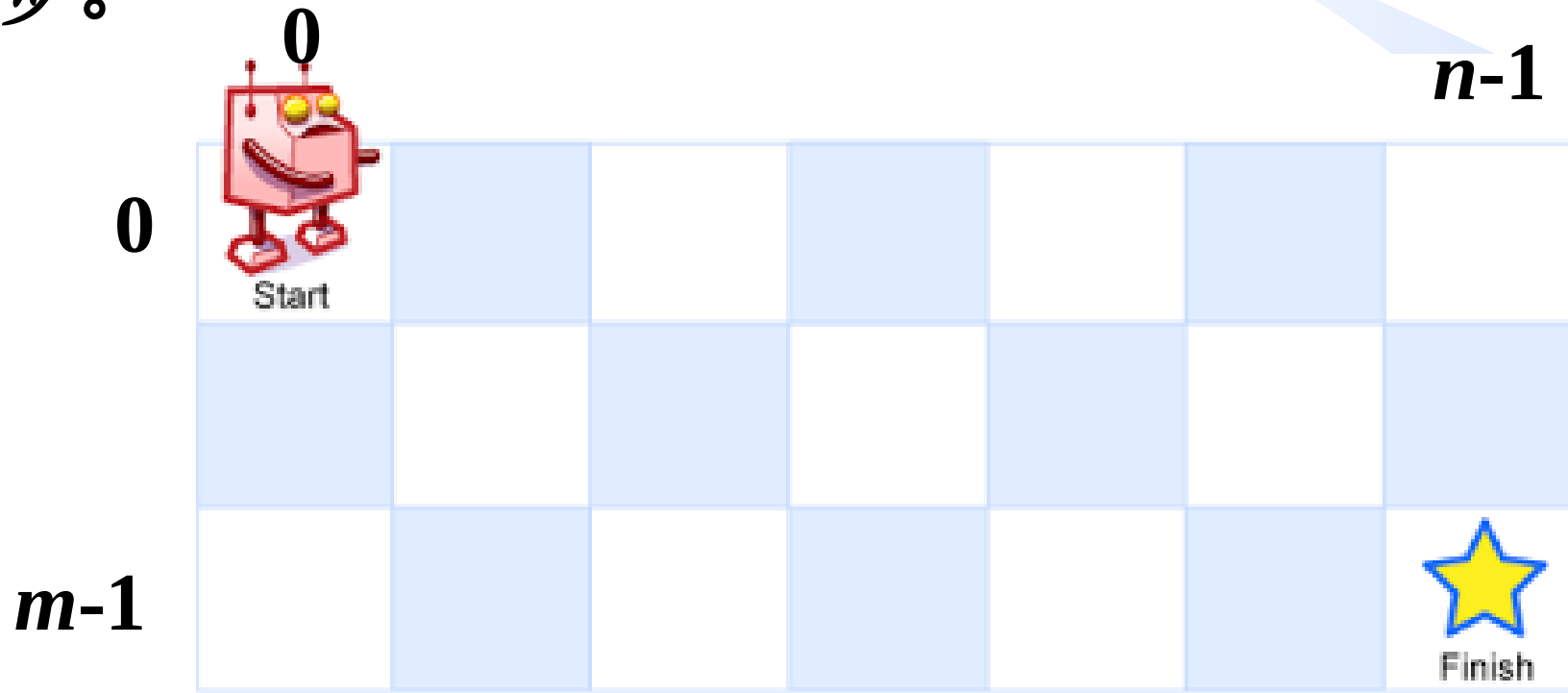
时间复杂度
 $O(mn)$
空间复杂度
 $O(n)$

源码之前
了无秘密



因为无障碍物，所以还有另一解法

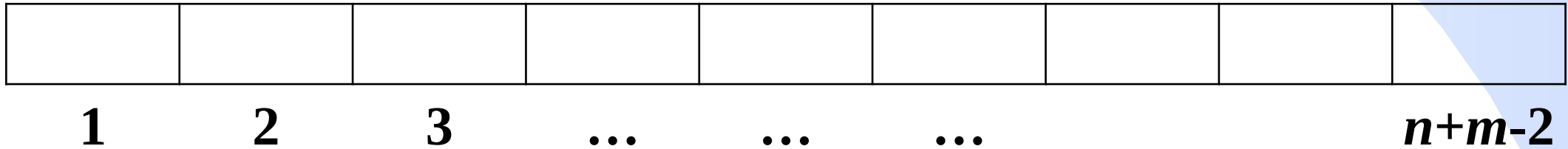
因为只能向右或向下走，想要从左上角到右下角，则向右走的步数一定是 $n-1$ 步，向下走的步数一定是 $m-1$ 步，合计走 $n+m-2$ 步。



另一解法

路径的总数 = 从 $n+m-2$ 步中选择 $m-1$ 步向下移动的方案数

$$C_{n+m-2}^{m-1}$$



如何高效计算 C_n^m

➤ 给定两个整数 n 和 m , 满足 $n \geq 1$ 且 $0 \leq m \leq n$, 编写程序计算 C_n^m , 输入数据保证最后结果在 $2^{31}-1$ 以内。【[POJ2249](#)】

$$C_n^m = \frac{n!}{m!(n-m)!} = \frac{n \cdot (n-1) \dots (n-m+3) \cdot (n-m+2) \cdot (n-m+1)}{m \cdot (m-1) \dots 3 \cdot 2 \cdot 1}$$

```
int Combination(int n, int m) {
    long long a = 1, b = 1;
    for(int i=n; i>=n-m+1; i--)
        a *= i;
    for(int i=m; i>=1; i--)
        b *= i;
    return a / b;
}
```



如何高效计算 C_n^m

➤ 给定两个整数 n 和 m , 满足 $n \geq 1$ 且 $0 \leq m \leq n$, 编写程序计算 C_n^m , 输入数据保证最后结果在 $2^{31}-1$ 以内。【[POJ2249](#)】

$$C_n^m = \frac{n!}{m!(n-m)!} = \frac{n \cdot (n-1) \dots (n-m+3) \cdot (n-m+2) \cdot (n-m+1)}{m \cdot (m-1) \dots 3 \cdot 2 \cdot 1}$$

```
int Combination(int n, int m) {
    long long a = 1, b = 1;
    for(int i=n; i>=n-m+1; i--)
        a *= i;
    for(int i=m; i>=1; i--)
        b *= i;
    return a / b;
}
```



中间结果溢出

如何高效计算 C_n^m

➤ 给定两个整数 n 和 m ，满足 $n \geq 1$ 且 $0 \leq m \leq n$ ，编写程序计算 C_n^m ，输入数据保证最后结果在 $2^{31}-1$ 以内。【[POJ2249](#)】

$$C_n^m = \frac{n!}{m!(n-m)!} = \frac{n \cdot (n-1) \dots (n-m+3) \cdot (n-m+2) \cdot (n-m+1)}{m \cdot (m-1) \dots 3 \cdot 2 \cdot 1}$$
$$= \frac{n}{m} \cdot \frac{n-1}{m-1} \cdot \frac{n-2}{m-2} \dots \frac{n-m+1}{1}$$

中间结果可能是
小数

如何高效计算 C_n^m

$$C_n^m = \frac{n \cdot (n-1) \dots (n-m+3) \cdot (n-m+2) \cdot (n-m+1)}{m \cdot (m-1) \dots 3 \cdot 2 \cdot 1}$$

$$= \frac{n-m+1}{1} \cdot \frac{n-m+2}{2} \cdot \frac{n-m+3}{3} \dots \frac{n-m+k}{k} \dots \frac{n-1}{m-1} \cdot \frac{n}{m}$$

$$C_{n-m+1}^1$$

中间结果一定
是整数

如何高效计算 C_n^m

$$C_n^m = \frac{n \cdot (n-1) \dots (n-m+3) \cdot (n-m+2) \cdot (n-m+1)}{m \cdot (m-1) \dots 3 \cdot 2 \cdot 1}$$

$$= \frac{n-m+1}{1} \cdot \frac{n-m+2}{2} \cdot \frac{n-m+3}{3} \dots \frac{n-m+k}{k} \dots \frac{n-1}{m-1} \cdot \frac{n}{m}$$

$$C_{n-m+2}^2$$

中间结果一定
是整数

如何高效计算 C_n^m

$$C_n^m = \frac{n \cdot (n-1) \dots (n-m+3) \cdot (n-m+2) \cdot (n-m+1)}{m \cdot (m-1) \dots 3 \cdot 2 \cdot 1}$$

$$= \frac{n-m+1}{1} \cdot \frac{n-m+2}{2} \cdot \frac{n-m+3}{3} \dots \frac{n-m+k}{k} \dots \frac{n-1}{m-1} \cdot \frac{n}{m}$$

$$C_{n-m+3}^3$$

中间结果一定
是整数

如何高效计算 C_n^m

$$C_n^m = \frac{n \cdot (n-1) \dots (n-m+3) \cdot (n-m+2) \cdot (n-m+1)}{m \cdot (m-1) \dots 3 \cdot 2 \cdot 1}$$

$$= \frac{n-m+1}{1} \cdot \frac{n-m+2}{2} \cdot \frac{n-m+3}{3} \dots \frac{n-m+k}{k} \dots \frac{n-1}{m-1} \cdot \frac{n}{m}$$

$$C_{n-m+k}^k$$

中间结果一定
是整数

如何高效计算 C_n^m

$$C_n^m = \frac{n-m+1}{1} \cdot \frac{n-m+2}{2} \cdot \frac{n-m+3}{3} \cdots \frac{n-m+k}{k} \cdots \frac{n-1}{m-1} \cdot \frac{n}{m}$$

```
int Combination(int n, int m) {  
    if (m == 0 || n == m) return 1;  
    if (m > n/2) m = n - m;  
    long long ans = 1; //通过i扫描分子, 通过j扫描分母  
    for (int i=n-m+1, j=1; i<=n; i++,j++)  
        ans=ans*i/j;  
    return ans;  
}
```

时间复杂度 $O(m)$
空间复杂度 $O(1)$

另一解法

一个机器人位于一个 $m \times n$ 网格的左上角，每次只能**向下**或者**向右**移动一步。机器人试图达到网格的右下角，编写程序计算总共有多少条不同的路径。【字节跳动、腾讯、华为、京东、苹果、微软、谷歌面试题 [LeetCode62](#)】

$$\text{路径的总数} = C_{n+m-2}^{m-1}$$

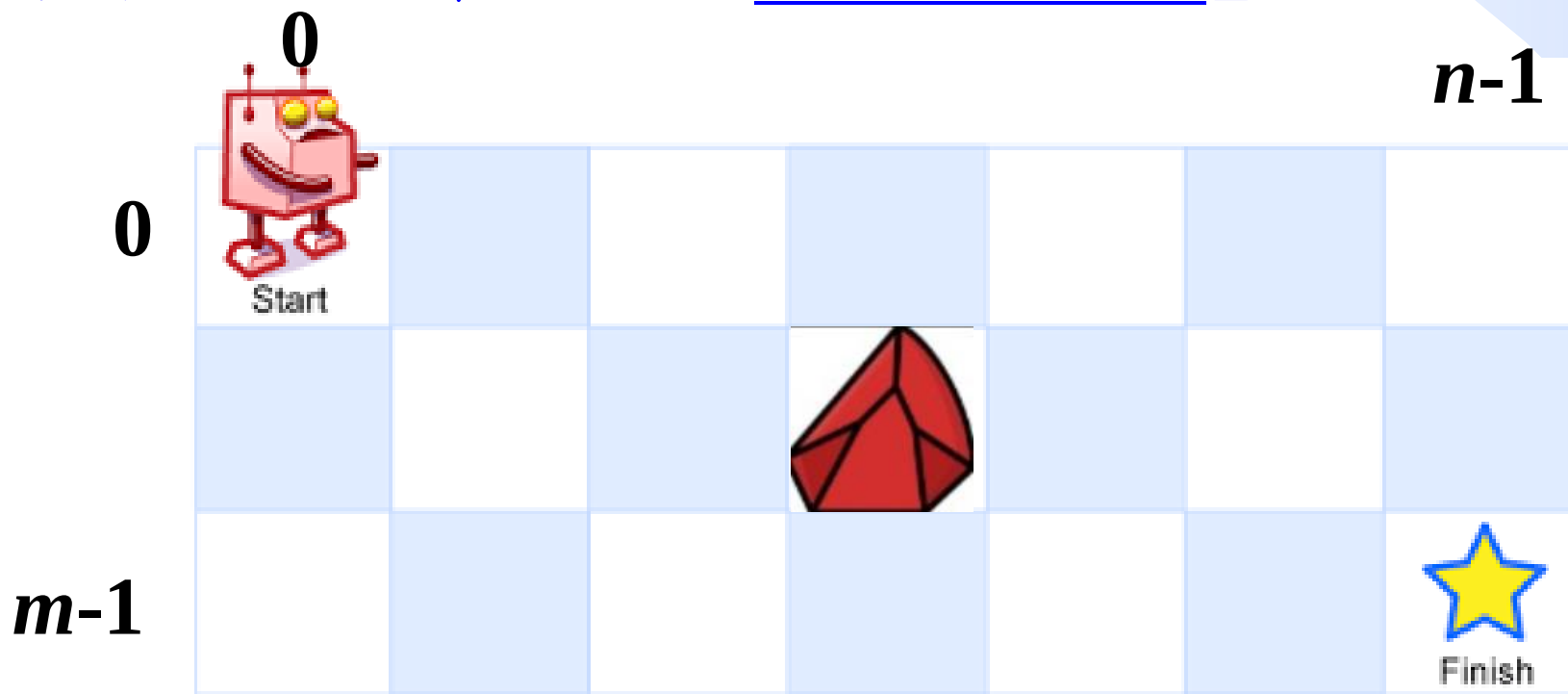
```
int uniquePaths(int m, int n) {  
    return Combination(n+m-2, m-1);  
}
```

时间复杂度
 $O(m)$

空间复杂度
 $O(1)$

课下思考

一个机器人位于一个 $m \times n$ 网格的左上角，每次只能**向下**或者**向右**移动一步。**网格中有障碍物**，机器人试图达到网格的右下角，编写程序计算总共有多少条不同的路径。【字节跳动、微软、谷歌、小红书面试题[LeetCode63](#)】



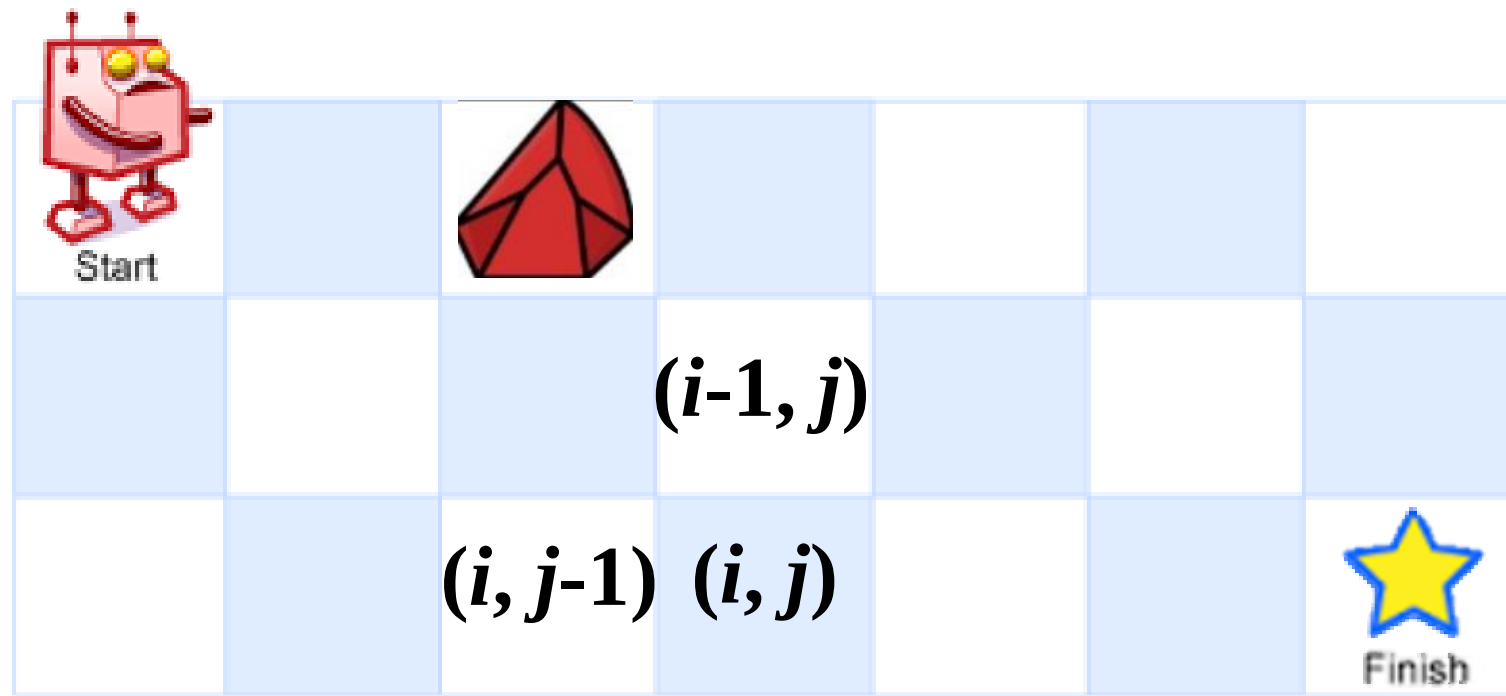
课下思考

令 $F(i, j)$ 表示起点到点 (i, j) 的路径条数

从起点到点 (i, j) 只有两种途径：

① 先到点 $(i-1, j)$ ，再向下走一步

② 先到点 $(i, j-1)$ ，再向右走一步



$$F(i, j) = \begin{cases} 0 & , map[i][j] = 1 \\ F(i, j-1) + F(i-1, j) & , i > 0 \text{ 且 } j > 0 \text{ 且 } map[i][j] = 0 \\ 1 & , i = 0 \text{ 且 } j = 0 \text{ 且 } map[i][j] = 0 \\ F(i, j-1) & , i = 0 \text{ 且 } j > 0 \text{ 且 } map[i][j] = 0 \\ F(i-1, j) & , i > 0 \text{ 且 } j = 0 \text{ 且 } map[i][j] = 0 \end{cases}$$

课下阅读（注意函数参数与LeetCode略有不同）

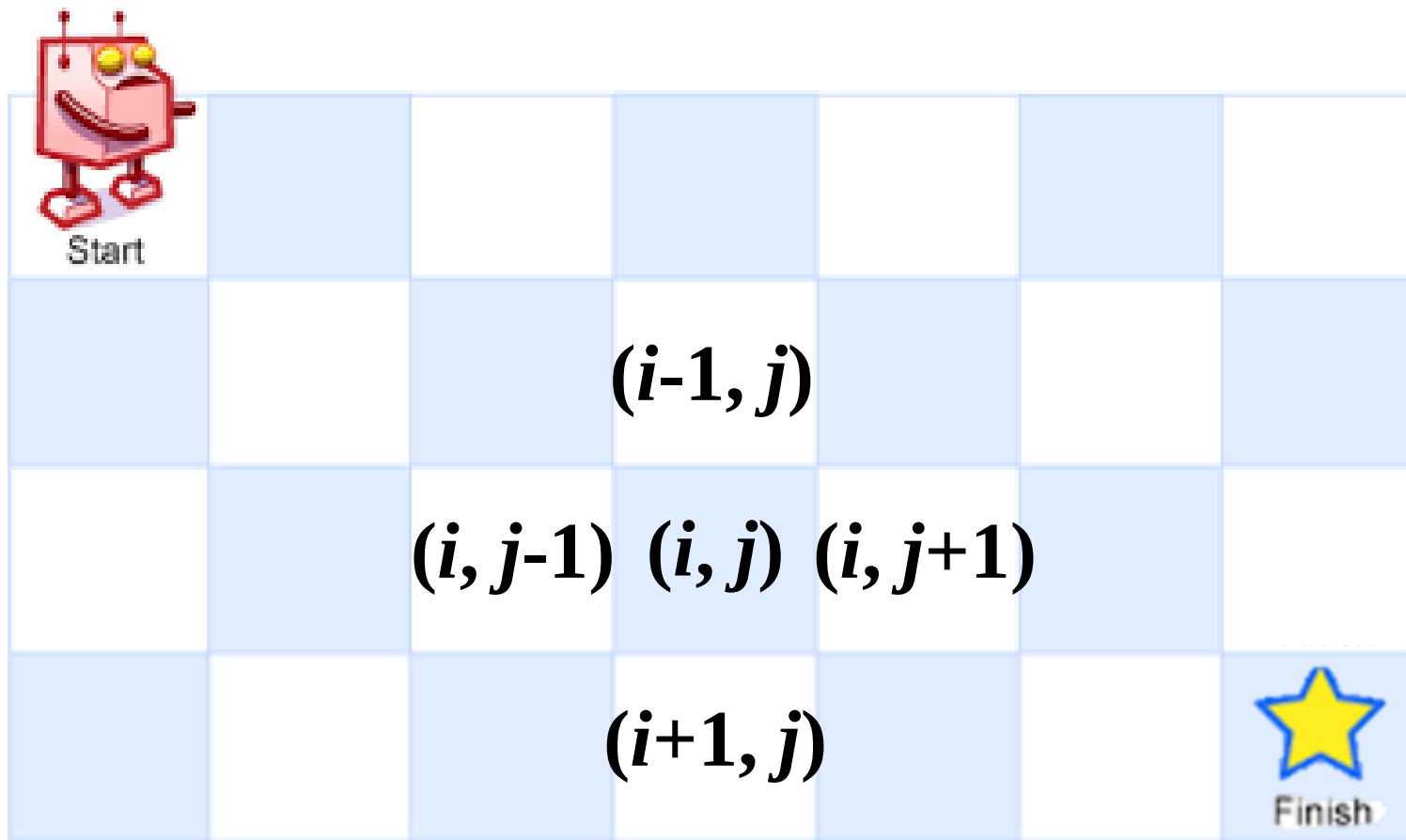
```
#define N 110
int PathsII(int map[N][N],int m,int n){
    int dp[N][N]={0};
    for(int i=0;i<m && map[i][0]==0;i++)
        dp[i][0]=1;
    for(int j=0;j<n && map[0][j]==0;j++)
        dp[0][j]=1;
    for(int i=1;i<m;i++)
        for(int j=1;j<n;j++)
            if(map[i][j]==1) dp[i][j]=0;
            else
                dp[i][j]=dp[i][j-1]+dp[i-1][j];
    return dp[m-1][n-1];
}
```

空间复杂度 $O(mn)$

```
#define N 110
int PathsII(int map[N][N],int m,int n){
    int dp[N]={0};
    for(int j=0;j<n && map[0][j]==0;j++)
        dp[j]=1;
    for(int i=1;i<m;i++)
        for(int j=0;j<n;j++)
            if(map[i][j]==1) dp[j]=0;
            else if(j>0)
                dp[j]=dp[j]+dp[j-1];
    return dp[n-1];
}
```

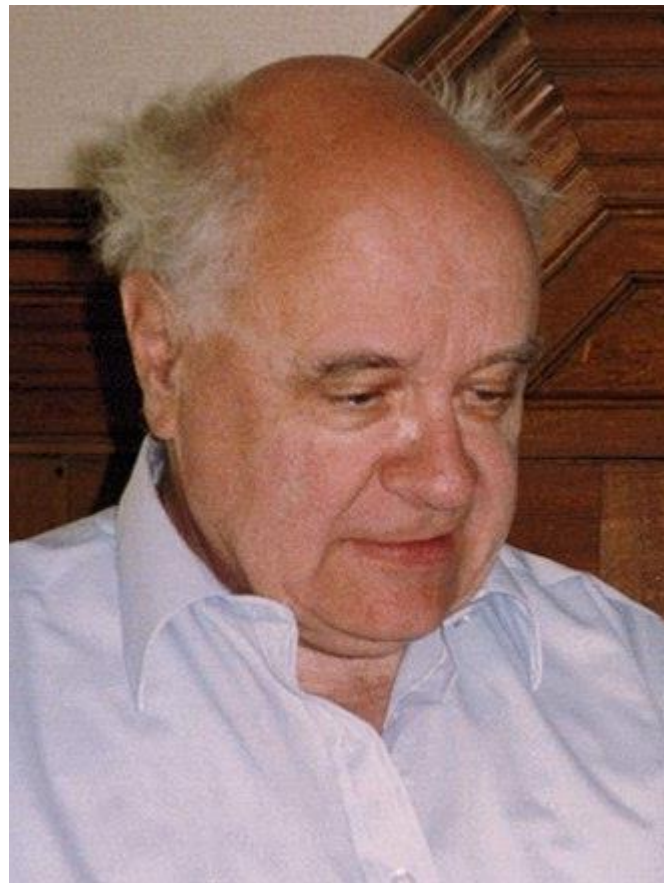
滚动数组优化
空间复杂度 $O(n)$

如果上、右、下、
左4个方向都能走，
还能使用动态规划
么？



最大子数组和

B



Ulf Grenander
(1923 - 2016)

斯德哥尔摩大学教授

布朗大学教授

瑞典科学院院士

美国科学院外籍院士

最大子数组和

给定一个含 n 个整数的数组，数组里可能有正数、负数和零。数组中连续的一个或多个元素组成一个子数组，每个子数组都有一个和。请设计算法求所有子数组之和的最大值。【字节跳动、阿里、微软、谷歌、腾讯、华为、百度、快手、携程、滴滴面试题，浙江大学考研复试机试[LeetCode53](#)、[HDU 1003](#)】

-2	5	3	-6	4	-8	6
----	---	---	----	---	----	---

1	-2	3	10	-4	7	-2	-5
---	----	---	----	----	---	----	----

最大子数组和

算法1: 遍历所有子数组。

```
int maxSubArray(int a[], int n){
```

```
    int maxsum =  $-\infty$ ;
```

```
    for(int i=0; i<n; i++){
```

```
        for(int j=i; j<n; j++){
```

```
            int sum = 0;
```

```
            for(int k=i; k<=j; k++)
```

```
                sum += a[k];
```

```
            if(sum > maxsum) maxsum = sum;
```

```
        }
```

```
    return maxsum;
```

```
}
```

C/C++中如何
表示无穷大?

//考察子数组a[i]...a[j]

//sum=a[i]+...+a[j]

1	-2	3	10	-4	7	-2	-5
---	----	---	----	----	---	----	----

C/C++ 表示正负无穷大的常用方法

方案1: `limits.h`头文件下的`INT_MAX`和`INT_MIN`宏

✓ 正无穷大: `INT_MAX` = $2^{31}-1$ = `0x7fffffff` = 2147483647

✓ 负无穷大: `INT_MIN` = -2^{31} = `0x80000000` = -2147483648

✓ 问题: $\infty + d \neq \infty$, $2\infty \neq \infty$

方案2: 自定义常量

00111111 00111111 00111111 00111111

✓ 正无穷大: `0x3f3f3f3f` = 1061109567

✓ 负无穷大: `0xc0c0c0c0` = -1061109568

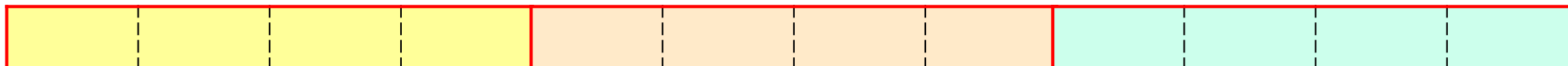
✓ 好处: ① 加上一个数或翻倍都不会溢出

② 将数组a中每个元素都初始化为无穷大

```
memset(a, 0x3f, sizeof(a));
```

```
memset(a, 0xc0, sizeof(a));
```

```
int a[3];
```



最大子数组和

算法1: 遍历所有子数组。

```
#include <limits.h>
```

```
int maxSubArray(int a[], int n){
```

```
    int maxsum = INT_MIN;
```

```
    for(int i=0; i<n; i++){
```

```
        for(int j=i; j<n; j++){           //考察子数组a[i]...a[j]
```

```
            int sum = 0;
```

```
            for(int k=i; k<=j; k++) //sum=a[i]+...+a[j]
```

```
                sum += a[k];
```

```
            if(sum > maxsum) maxsum = sum;
```

```
        }
```

```
    return maxsum;
```

```
}
```

时间复杂度 $O(n^3)$

1	-2	3	10	-4	7	-2	-5
---	----	---	----	----	---	----	----

最大子数组和

算法2: 利用 $\text{sum}[i..j] = \text{sum}[i..j-1] + a[j]$ 优化。

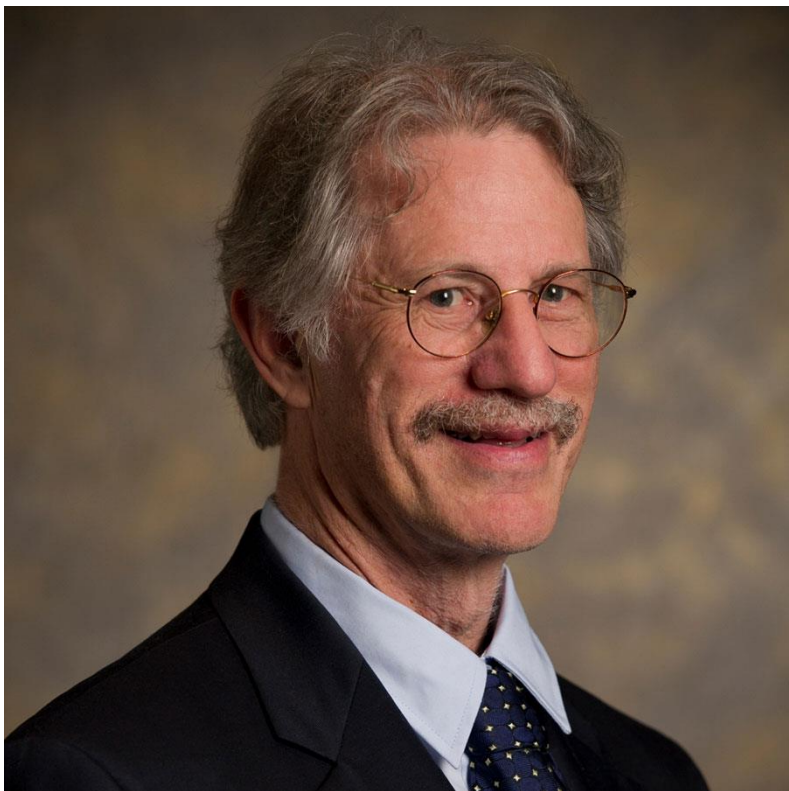
```
int maxSubArray(int a[], int n){  
    int maxsum = INT_MIN;  
    for(int i=0; i<n; i++){  
        int sum = 0;  
        for(int j=i; j<n; j++) {  
            sum += a[j];  
            if(sum > maxsum) maxsum = sum;  
        }  
    }  
    return maxsum;  
}
```

时间复杂度 $O(n^2)$

1	-2	3	10	-4	7	-2	-5
---	----	---	----	----	---	----	----

最大子数组和——线性时间算法

B

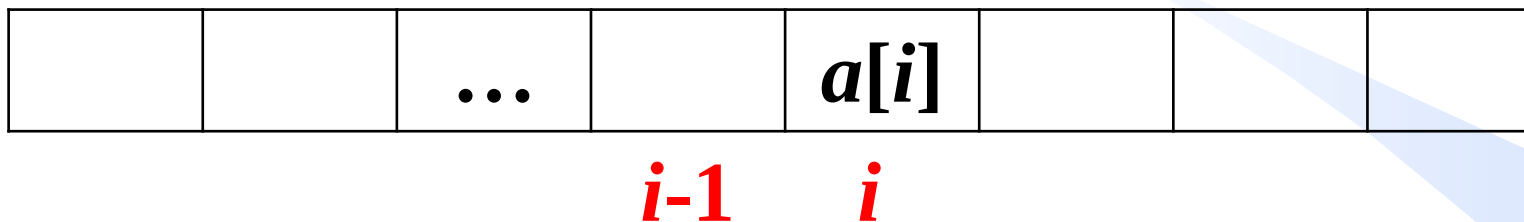


Joseph B. Kadane
(1941 -)

卡内基梅隆大学教授
美国艺术与科学院院士

最大子数组和

$f(i)$ 表示所有以位置 i 结尾的子数组的最大和。



✓ $f(i-1)$ 表示所有以位置 $i-1$ 结尾的子数组的最大和。

✓ 以位置 i 结尾的最大子数组一定包含 $a[i]$ ；但包不包含 $a[i]$ 前面的元素？不一定，可能包含也可能不包含。

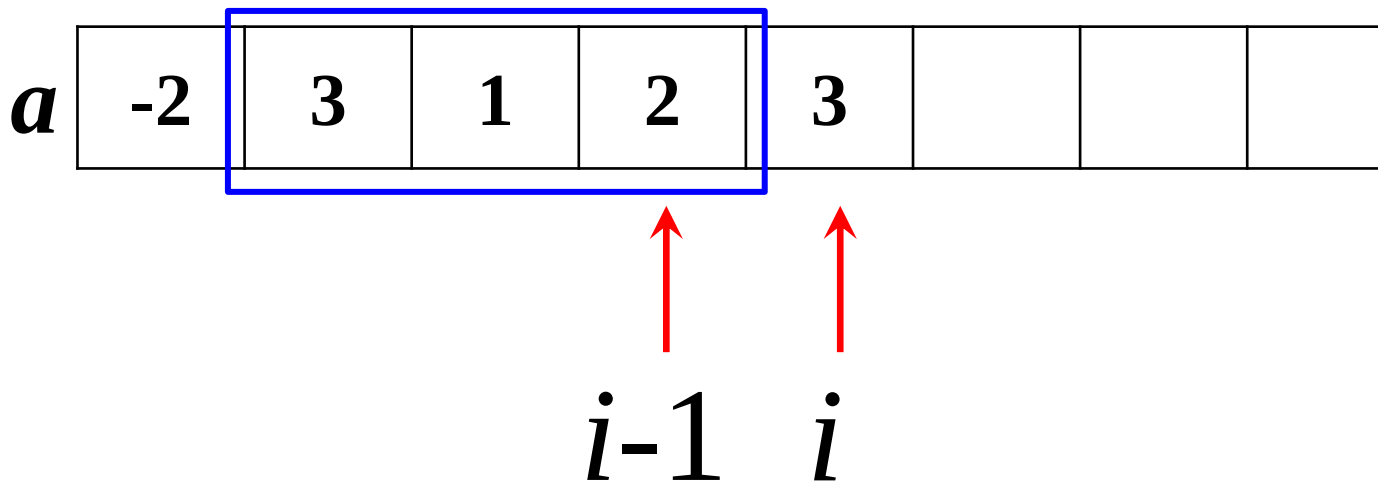
最大子数组和

情况1: 当 $f(i-1) > 0$ 时

✓ 以位置 i 结尾的最大子数组 = 以位置 $i-1$ 结尾的最大子数组 + $a[i]$

$$f(i) = f(i-1) + a[i], \quad f(i-1) > 0$$

$$f(i-1) = 6$$

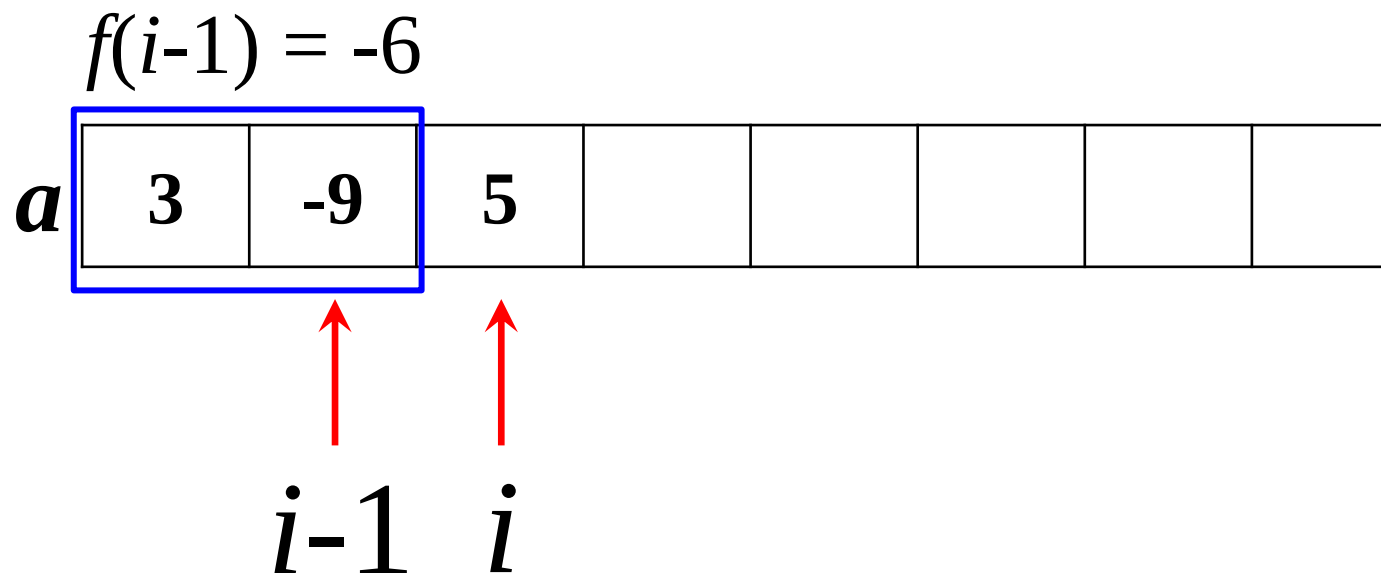


最大子数组和

情况2: 当 $f(i-1) \leq 0$ 时, 则 $a[i] + f(i-1) \leq a[i]$

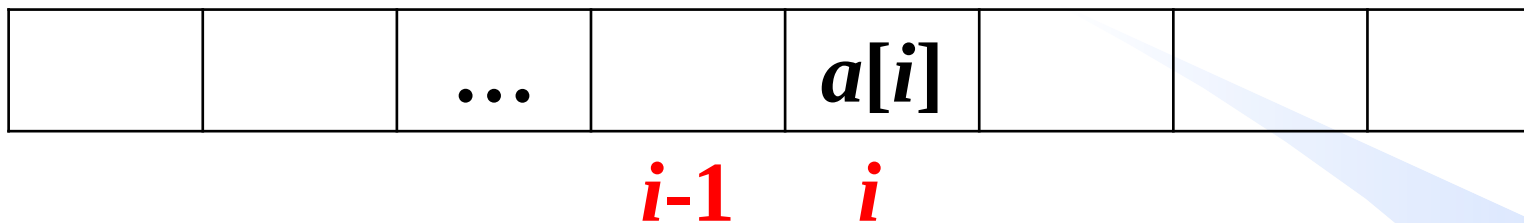
✓ 以位置 i 结尾的最大子数组 = $a[i]$

$$f(i) = a[i], \quad f(i-1) \leq 0$$



最大子数组和

$f(i)$ 表示以位置*i*结尾的最大子数组的和。



$$f(i) = \begin{cases} f(i-1) + a[i] & , f(i-1) > 0 \\ a[i] & , f(i-1) \leq 0 \mid i = 0 \end{cases}$$

所有子数组的最大和 = $\max_i f(i)$

最大子数组和

```
const int maxn = 1e5+10;
int maxSubArray(int a[], int n){
    int f[maxn]; f[0] = a[0];
    int maxsum = a[0];
    for(int i=1; i<n; i++) {
        if (f[i-1] > 0) f[i] = f[i-1] + a[i];
        else f[i] = a[i];
        if (f[i] > maxsum) maxsum = f[i];
    }
    return maxsum;
}
```

时间复杂度 $O(n)$
空间复杂度 $O(n)$

$$f(i) = \begin{cases} f(i-1) + a[i] & , f(i-1) > 0 \\ a[i] & , f(i-1) \leq 0 \mid i = 0 \end{cases}$$

最大子数组和——空间优化

B

```
const int maxn = 1e5+10;
int maxSubArray(int a[], int n){
    int f = a[0];
    int maxsum = a[0];
    for(int i=1; i<n; i++) {
        if (f > 0) f = f + a[i];
        else f = a[i];
        if (f > maxsum) maxsum = f;
    }
    return maxsum;
}
```

时间复杂度 $O(n)$
空间复杂度 $O(1)$

$$f(i) = \begin{cases} f(i-1) + a[i] & , f(i-1) > 0 \\ a[i] & , f(i-1) \leq 0 \mid i = 0 \end{cases}$$

课下思考

找出最大子数组，若存在多个最大子数组，则返回长度最长的，输入数据保证最长的最大子数组只有一个。【大厂面试题，牛客JZ85】

```
int* maxSubArray(int a[], int n, int &subArraySize){ //参数类型与牛客不同
    int f = a[0], maxsum = a[0];
    int start = 0, end = 0, maxstart = 0, maxend = 0;
    for(int i = 1; i < n; i++) {
        if (f >= 0) f = f + a[i];           //起点是f(i-1)的起点
        else { f = a[i]; start = i; }      //起点就是i
        end = i;
        if(f > maxsum || (f==maxsum && (end-start)>(maxend-maxstart))){
            //若有多个子数组满足条件，返回最长的
            maxsum = f; maxstart = start; maxend = end;
        }
    }
    subArraySize = maxend - maxstart + 1;
    return &a[maxstart]; //返回以maxstart为起点，长度为subArraySize的子数组
}
```

时间复杂度 $O(n)$
空间复杂度 $O(1)$

拓展：最大子数组乘积

给定一个整数数组，请找出数组中乘积最大的非空连续子数组（该子数组中至少包含一个数字），并返回该子数组所对应的乘积。【大厂面试题 [LeetCode152](#)】

-2	4	0	3	2	8	-1
----	---	---	---	---	---	----

			...	$a[i]$			
				i			

$f(i)$ 表示以位置 i 结尾的最大子数组的乘积。

$g(i)$ 表示以位置 i 结尾的最小子数组的乘积。

$$f(i) = \max\{a[i], f(i-1) \times a[i], g(i-1) \times a[i]\}$$

$$g(i) = \min\{a[i], f(i-1) \times a[i], g(i-1) \times a[i]\}$$

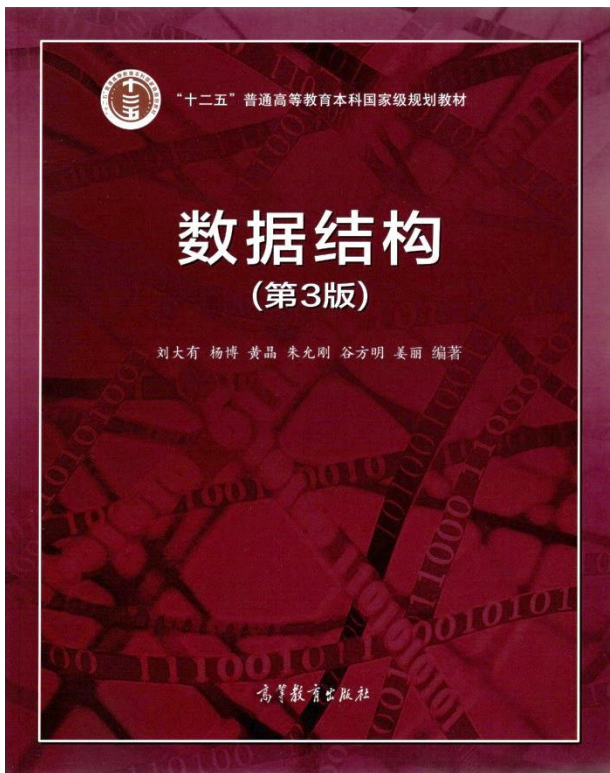
最大子数组乘积——课下阅读

```
int maxProduct(int a[], int n){  
    int f=a[0], g=a[0], maxproduct=a[0];  
    for(int i=1;i<n;i++){  
        int fa=f*a[i], ga=g*a[i];  
        f=max(a[i], fa, ga);  
        g=min(a[i], fa, ga);  
        if(f>maxproduct) maxproduct=f;  
    }  
    return maxproduct;  
}
```

时间复杂度 $O(n)$
空间复杂度 $O(1)$

```
int max(int a, int b, int c){  
    int maxval=a;  
    if(b>maxval) maxval=b;  
    if(c>maxval) maxval=c;  
    return maxval;  
}
```

```
int min(int a, int b, int c){  
    int minval=a;  
    if(b<minval) minval=b;  
    if(c<minval) minval=c;  
    return minval;  
}
```



数组与矩阵

- 数组存储与寻址
- 特殊矩阵的压缩存储
- 稀疏矩阵的压缩存储
- 动态规划初探
- **子集生成**

数据之法
结构之美
算法之道

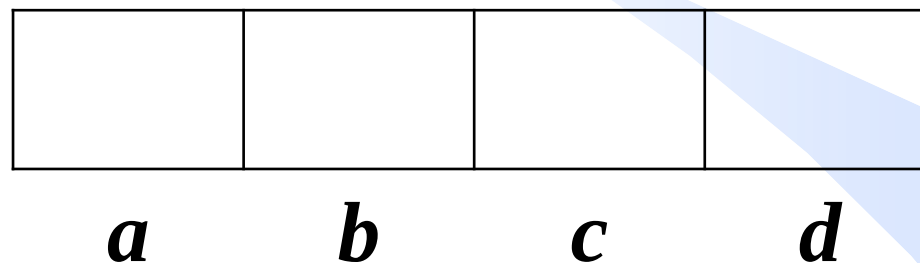
zhuyungang@jlu.edu.cn

子集生成

输出一个集合的幂集，如集合 $\{a, b, c, d\}$ 的幂集如下。【百度、字节跳动、华为、快手、谷歌面试题】

```
{ }
{ a }
{ b }
{ a b }
{ c }
{ a c }
{ b c }
{ a b c }
{ d }
{ a d }
{ b d }
{ a b d }
{ c d }
{ a c d }
{ b c d }
{ a b c d }
```

二进制数 $\leftrightarrow$ 子集



二进制数加计数器

课下阅读

```
void PowerSet(char s[], int n){
    int a[maxsize] = { 0 };
    while (a[n] == 0) {
        printf("{ ");
        for(int i=0; i<n; i++)
            if (a[i] == 1)
                printf("%c ", s[i]);
        printf("}\n");
        AddOne(a, n);
    }
}
```

```
const int maxsize = 10;
void AddOne(int a[], int n) {
    int i = 0;
    while (a[i] != 0)
        a[i++] = 0;
    a[i] = 1;
}
```

```
int main() {
    char s[maxsize]={'a','b','c','d',
'e','f','g'};
    int n=4;
    PowerSet(s, n);
    return 0;
}
```

类似题目
[洛谷B3622](#)

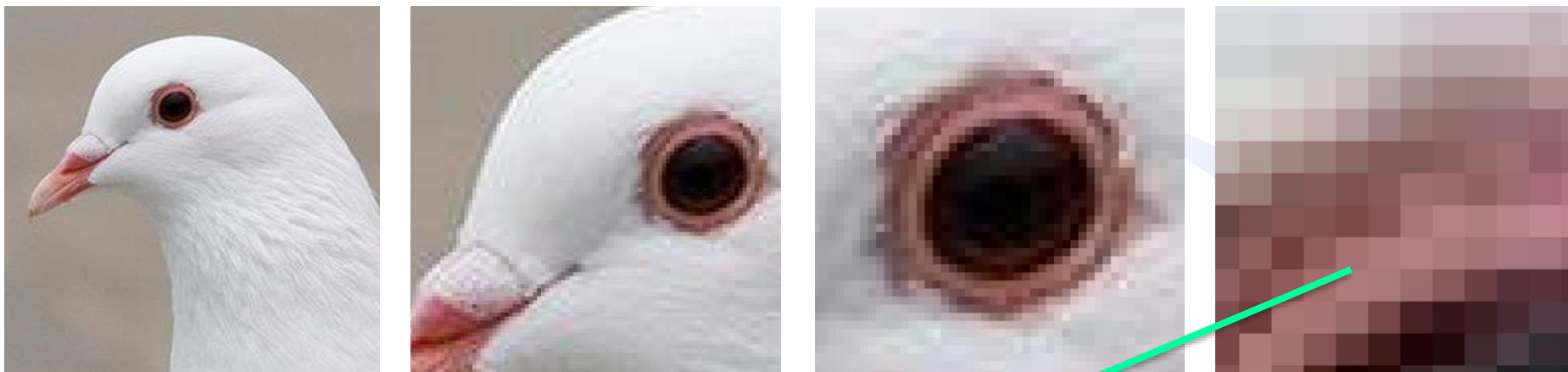
课下思考

请编写程序以字典序输出一个集合的所有子集。输入为一个正整数 n ($n \leq 15$) 表示集合包含的元素个数。约定元素的编号为 $a, b, c, d \dots$ 。若 $n=1$ ，集合元素为 a ；若 $n=3$ ，集合元素为 a, b, c ；若 $n=4$ ，集合元素为 a, b, c, d ；以此类推。【吉林大学2024年保研夏令营/预推免复试机试题】

输入	输出
3	{} {a} {a,b} {a,b,c} {a,c} {b} {b,c} {c}

输入	输出
4	{} {a} {a,b} {a,b,c} {a,b,c,d} {a,b,d} {a,c} {a,c,d} {a,d} {b} {b,c} {b,c,d} {b,d} {c} {c,d} {d}

矩阵应用-图像处理



$$\begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} & \cdots & a_{1,n} \\ a_{2,1} & a_{2,2} & a_{2,3} & \cdots & a_{2,n} \\ a_{3,1} & a_{3,2} & a_{3,3} & \cdots & a_{3,n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m,1} & a_{m,2} & a_{m,3} & \cdots & a_{m,n} \end{bmatrix}$$

黑白图像：灰度值
彩色图像：RGB值



自愿性质OJ练习题

- ✓ [LeetCode 70](#) (跳台阶)
- ✓ [LeetCode 53](#) (最大子数组和)
- ✓ [HDU 1003](#) (最大子数组和高级版)
- ✓ [牛客JZ85](#) (最大子数组和高级版)
- ✓ [LeetCode 152](#) (最大子数组乘积)
- ✓ [LeetCode 62](#) (机器人走迷宫路线数)
- ✓ [LeetCode 63](#) (机器人走迷宫路线数, 迷宫带障碍物)
- ✓ [洛谷 B3622](#) (子集生成)
- ✓ [POJ 2249](#) (求组合数)
- ✓ [LeetCode 27](#) (数组删除元素)